1.

A food delivery app confirms payment and shows the order as “Placed Successfully.”
However, the restaurant dashboard never receives the order, while the customer’s money is already deducted.

Customer support can see the payment record but no order record in the admin panel.

As an SQA engineer:
How would you trace the complete flow from payment gateway to order creation?
Which services, logs, APIs, and failure points would you validate?

Ans: Assumptions Based on the Scenario

1. Payment success is handled separately from order creation.

2.The app shows Placed Successfully immediately after payment confirmation, not after order DB confirmation.

3. Payment record is saved successfully in the Payment Service.

4. Order creation depends on a separate Order Service/API call.

5. The Order Service failed (API error, timeout, validation issue, or DB failure).

6. No proper transaction rollback exists between payment and order creation.

7. There is no distributed transaction ensuring atomicity.

8. Event/message queue failure may have prevented order from reaching restaurant dashboard.

9. Admin panel and restaurant dashboard read from the Order database, but no order record was created.

10. System lacks proper reconciliation logic for “Payment Success but Order Not Created” cases.

Test Ideas for This Scenario (Payment Success but No Order Created)

1. End-to-End Flow Tests

- Verify order is created in DB after successful payment.
- Validate Order ID is generated and linked with Payment Transaction ID.
- Confirm restaurant dashboard receives the order after payment success.
- Check admin panel shows both payment record and order record.

2. API Validation Tests

- Test Payment Success → Order Creation API trigger.
- Validate response codes between Payment Service and Order Service.
- Simulate Order Service failure (500/timeout) after payment success.
- Verify idempotency (same payment callback should not create duplicate orders).

3. Failure & Negative Tests

- Simulate DB failure during order creation.
- Simulate message queue/event failure to restaurant dashboard.
- Simulate webhook delay from payment gateway.
- Test network interruption after payment success.
- Verify system behavior if restaurant is closed after payment.

4. Data Consistency Tests

- Validate payment - id is mapped correctly with order -id.
- Check system handling for Payment Success but Order Not Created.
- Verify auto-refund or reconciliation process triggers correctly.
- Confirm no orphan payment records remain

5. Logging & Monitoring Tests

- Validate Correlation/Trace ID across services.
- Check logs for payment → order flow.
- Verify alerts trigger for payment/order mismatch cases.

6. UI/UX Validation

- Ensure “Placed Successfully” is shown only after order confirmation.
- Validate proper error message if order creation fails after payment.
- Confirm order appears in customer order history

Edge Cases for This Scenario

Payment Related Edge Cases

1. Payment succeeds but Order Service API times out.
2. Payment succeeds but webhook/callback is delayed.
3. Duplicate payment callback received .
4. Payment marked success but status later reversed by gateway.
5. Partial payment / split payment scenario.
6. Payment success response received but DB write for payment fails.

Order Creation Edge Cases

7. Order creation fails due to DB constraint/validation error.
8. Restaurant becomes closed after payment but before order creation.
9. Item goes out of stock after payment confirmation.
10. Invalid restaurant/branch ID mapping.
11. Order created with wrong status.
12. Duplicate order attempt for same payment ID.

Integration / Messaging Edge Cases

13. Event/message queue failure while sending order to restaurant dashboard.
14. Consumer service (restaurant dashboard) is down.
15. Read replica lag causing order not visible immediately.
16. Cache not refreshed, order exists but not shown.

System & Concurrency Edge Cases

17. Network interruption after payment success response.
18. High traffic causing race condition between services.
19. Retry mechanism creates duplicate records.
20. Distributed transaction failure.

Reconciliation & Recovery Edge Cases

21. No auto-refund triggered when order creation fails.
22. Orphan payment record without linked order ID.
23. Manual reconciliation fails to detect mismatch.
24. Order created but not linked to customer account

Risks in This Scenario (Payment Success but No Order Created)

Financial Risk – Customer is charged but order is not placed, leading to refund liability and revenue loss.

Customer Trust Risk – Loss of customer confidence due to money deduction without service delivery.

Brand Reputation Risk – Negative reviews, social media complaints, and reduced user retention.

Operational Risk – Support team workload increases due to manual complaint handling and refunds.

Data Integrity Risk – Payment and order data mismatch.

Compliance Risk – Potential violation of financial regulations or consumer protection laws.

Duplicate Order Risk – Retry mechanisms may create duplicate orders once issue is fixed.

System Reliability Risk – Indicates weak transaction handling between Payment and Order services.

Reconciliation Risk – Lack of automated detection for Payment Success but No Order cases.

Revenue Leakage Risk – Failed orders without proper tracking may cause accounting inconsistencies.

2.

After releasing a new mobile app version, users report they can log in successfully when connected to Wi-Fi but fail when using mobile data.

The backend team says the authentication service is working fine and sees no errors in normal logs.

How would you design tests to isolate whether the issue is related to network configuration, timeout handling, API security rules, or client-side logic?

Ans: Assumptions Based on the Scenario

1. Authentication service is reachable and functional when requests originate from trusted/internal networks (e.g., Wi-Fi).

2. Mobile data traffic may traverse different network paths

3. The backend logs only successful or fully received requests, so failed mobile attempts may not be reaching the service.

4. The app version is identical in both scenarios, with no feature flags or environment switching between Wi-Fi and mobile data.

5. API endpoints, domains, and certificates are publicly accessible and not intentionally restricted to specific IP ranges.

6. Mobile carriers may enforce stricter timeout, DNS, or TLS policies compared to Wi-Fi networks.

7. No recent changes were made to API gateway, WAF, or security rules that selectively block certain IP ranges or traffic types.

8. The client app handles network errors and timeouts explicitly, but error reporting may not clearly distinguish failure causes.

9. SSL/TLS certificates are valid and not pinned incorrectly in a way that behaves differently across networks.

10. The issue is reproducible across multiple users and carriers, not limited to a single device or SIM provider.

Test Ideas for This Scenario

1. Network configuration (DNS / IPv6 / carrier routing)

- **Network matrix:** Reproduce on multiple Wi-Fi networks and multiple carriers/SIMs on the same app build/device.
- **Hotspot control:** Use mobile data hotspot test login from another device, compare with normal Wi-Fi.
- **IPv4/IPv6 check:** Force IPv4-only vs IPv6-only (or test on an IPv6-capable carrier) to detect routing/TLS issues.
- **DNS swap:** Override DNS on mobile data to rule out carrier DNS problems.
- **Reachability verification:** From mobile data, confirm the auth domain is reachable.

2. Timeout and retry handling

- **Network throttling:** Simulate high latency/packet loss and confirm whether login failures correlate with slow networks.

- **Timeout threshold test:** Measure request duration and identify if failures occur at a consistent time boundary.

- **Retry behavior:** Validate retries/backoff and cancellation handling (app background/foreground, network switching mid-request).

3. API security rules (WAF / gateway / IP restrictions / rate limits)

- **Edge visibility:** Check CDN/WAF/API gateway logs for failed mobile attempts.

- **IP/rule validation:** Capture mobile public IP and confirm it isn't geo/IP blocked or flagged by bot/rate-limit rules.

- **Header comparison:** Compare request headers and TLS parameters between Wi-Fi success and mobile failure.

4. Client-side logic defects

- **Request/response capture:** Collect full HTTP traces on Wi-Fi vs mobile (URL, headers, body, status, payload) and diff them.

- **Connectivity gating:** Confirm the app actually calls the login endpoint on mobile data.

- **Error mapping:** Verify "login failed" matches actual network/API errors.

Edge Cases

Carrier NAT Impact – Authentication requests blocked or rate-limited due to shared public IP ranges on mobile networks.

IPv6 / NAT64 Compatibility – Login fails on IPv6-only mobile networks while functioning on IPv4 Wi-Fi.

WAF / Geo-IP Restrictions – Security rules unintentionally blocking carrier IP ranges or specific regions.

TLS Handshake Variations – Certificate validation or TLS version mismatch occurring only over cellular routing paths.

High Latency Handling – Client-side timeout thresholds too low for slower mobile data conditions.

Packet Loss / Network Instability – Intermittent failures due to weak signal or tower switching during authentication.

Network Transition Scenario – Session failure when switching between Wi-Fi and mobile during login request.

Rate Limiting Threshold – Multiple retry attempts on mobile triggering API rate limits (429 responses).

Connectivity Detection Logic – Application incorrectly classifying mobile data as offline state.

Token Persistence Issue – Successful authentication response received but access token not stored reliably under mobile conditions.

Key Risks Based on the Scenario:

1. **Business Risk**

- **User Login Failure on Mobile Data** → High risk of user drop-off, negative reviews, and revenue impact.
- **Production Defect in New Release** → Brand reputation damage and loss of customer trust.
- **Increased Support Tickets** → Operational cost and support overload.

2. **Technical Risk**

- **Network Compatibility Gaps** → Application not fully compatible with IPv6, carrier NAT, or mobile routing.
- **Timeout Misconfiguration** → Client timeout settings unsuitable for high-latency mobile environments.
- **Security Rule Misconfiguration** → WAF, rate limits, or IP restrictions unintentionally blocking valid users.

- **Unlogged Failures** → Requests not reaching backend, making root cause detection difficult.

3. **Security Risk**

- **Overly Strict API Protection** → Legitimate traffic blocked due to aggressive bot/IP filtering.
- **Certificate/TLS Issues** → Handshake failures leading to authentication breakdown.

4. **User Experience Risk**

- **Inconsistent Behavior** → App works on Wi-Fi but fails on mobile, causing confusion.
- **Incorrect Error Messaging** → Users receive generic “Login Failed” without actionable feedback.

5. **Release Management Risk**

- **Insufficient Network Coverage Testing** → Lack of real-device, real-carrier validation before release.
- **Environment/Configuration Drift** → Differences between staging and production gateway/security settings.

Overall Risk Level: High

Because login is a critical entry-point feature, failure on mobile networks directly affects accessibility, retention, and business continuity.

3.

During a massive flash sale, thousands of users add the same limited-stock product to their carts.

Some users manage to complete payment, but later receive cancellation messages saying “Out of stock,” while others who paid later successfully receive the product.

What concurrency issues could be happening?
How would you test inventory locking, race conditions, and transaction atomicity?

Ans: Assumptions – Flash Sale Concurrency Scenario

1. Limited Inventory
The product has fixed, finite stock that must never drop below zero.

2. High Concurrency
Thousands of users perform add-to-cart, checkout, and payment operations simultaneously.

3. Distributed Architecture
Inventory, order, and payment services operate independently and may communicate asynchronously .

4. Non-Atomic Operations
Inventory validation, order creation, and payment confirmation may occur in separate transactions.

5. Reservation Policy
Stock may not be reserved at add-to-cart and could be reserved only at checkout or payment confirmation.

6. Eventual Consistency
Inventory reads may come from cache or replicas and can be temporarily stale under heavy load.

7. Retry & Idempotency
Payment callbacks and background jobs may retry, potentially causing duplicate processing if not idempotent.

8. System Invariant
The total number of confirmed (paid and fulfilled) orders must never exceed available stock.

Flash Sale – Concurrency Test Ideas

1. Inventory Locking

- Simulate thousands of concurrent checkouts → ensure confirmed orders ≤ stock.
- Verify stock never goes negative.
- Test reservation expiry and release.
- Test duplicate reservation attempts → only one succeeds.

2. Race Conditions

- Trigger simultaneous payment successes → no overselling.
- Send duplicate/out-of-order payment webhooks → ensure idempotency.
- Test concurrent confirm vs cancel → final state must be consistent.
- Test stale cache reads under load.

3. Transaction Atomicity

- Simulate failure between payment and stock deduction → no partial updates.
- Retry order finalization multiple times → stock deducted only once.
- Stress test until stock = 0 → system must stop accepting orders.

Edge Cases – Flash Sale Inventory & Payment Concurrency

- **Stock = 1 boundary**: thousands attempt checkout → exactly 1 fulfilled, no negative stock.
- **Last-unit race**: two checkouts reserve/deduct at same time → only one succeeds.
- **Payment success but reservation expired** → correct cancel/refund, no stock leak.
- **Payment webhook duplicated** (same transaction sent multiple times) → idempotent; no double deduction.
- **Webhook out-of-order**: success arrives after failed/cancel → final state consistent.
- **Delayed webhook**: success arrives minutes later after stock is sold → handled via reservation rule + compensation.
- **Client retries / double-click pay**: multiple payment attempts for same order → charge once, finalize once.
- **Multiple tabs/devices**: same user checks out twice → only one order confirmed/reserved.
- **Partial failure**: payment captured but DB update fails (timeout/rollback) → compensation triggers, no orphan paid order.

- **Inventory service timeout** during checkout → order not confirmed without reservation; safe retry.
- **Deadlock/lock wait timeout** under heavy load → graceful failure + retry policy, no inconsistent states.
- **Cache stale / replica lag** shows in stock after depletion → final commit must still reject safely.
- **Cancel vs fulfill race**: cancellation worker and fulfillment worker run together → single final status.
- **Refund failure** after oversell cancellation → order stays refund pending, no reship/duplicate fulfill.
- **Message queue at-least-once delivery**: finalize event reprocessed → exactly-once effect via idempotency key.
- **Backorder/preorder flag misapplied** during sale → ensure SKU rules don't allow unintended oversell

Key Risks – Flash Sale Concurrency

1. **Overselling** – Stock goes below zero due to race conditions.

2. **Unfair allocation** – Early payers canceled, later payers fulfilled.

3. **Financial loss** – Captured payments require refunds/chargebacks.

4. **Data inconsistency** – Order, payment, and inventory states mismatch.

5. **Duplicate processing** – Retries/webhooks cause double deduction or double confirmation.

6. **Customer trust damage** – Paid but out of stock impacts brand reputation.

7. **System instability** – Lock contention, timeouts, and cascading failures under peak load.

4.

A user updates their email address in their profile successfully.
The UI shows the new email everywhere.

However, password reset links and promotional emails are still being sent to the old email address days later.

Which systems could be out of sync?
How would you validate data consistency across databases, caches, and third-party email services?

Ans: Assumptions Based on the Scenario

1. The email update functionality successfully writes the new email address to the primary user profile database, and the update confirmation indicates a successful transaction.

2. The UI retrieves and displays user profile information from the primary profile service or read model, which reflects the latest updated email.

3. The system architecture includes separate services for profile management, authentication, and email delivery, and these services may maintain independent data stores.

4. Password reset emails are generated by an authentication service, which may retrieve the user's email from its own database, cache, or token store rather than the primary profile database.

5. Promotional or marketing emails are managed through a third-party Email Service Provider or CRM system, which maintains its own contact list and may require explicit synchronization when profile data changes.

6. Email address updates are propagated to dependent systems through asynchronous messaging which may introduce delays or synchronization failures.

7. The system may use caching layers or read replicas, which can temporarily retain stale email data if cache invalidation or replication is delayed.

8. Previously scheduled, queued, or campaign-based emails may continue to use the old email address if the recipient list was generated before the update occurred.

9. A unique user ID is assumed to be the primary identifier across systems, and email is treated as a mutable attribute.

10. Logging and monitoring mechanisms are assumed to be available to trace which data source was used at the time of email generation.

Key QA Test Ideas

1. **Profile Update Source-of-Truth**
    - Change email → verify DB record updated (by user id) and old email removed/marked inactive.
2. **UI vs Backend Consistency**
    - Refresh UI, logout/login, different devices/browser → confirm UI always shows new email and never reverts.
3. **Password Reset Uses New Email**
    - After change, trigger “Forgot password” → confirm reset email is delivered to new email only.
4. **Old Email Should Not Receive Reset**
    - Try password reset using old email (if supported) → ensure system blocks or routes to new email, and **no email goes to old.**
5. **Immediate vs Delayed Behavior**
    - Repeat password reset at intervals (immediately, 15 min, 1 hr, 24 hr) → ensure no delayed sends to old email.
6. **Cache/Replica Staleness**
    - After change, trigger reset from multiple app nodes/environments (if possible) → ensure no node sends to old email.
7. **Event Propagation Validation**
    - Confirm email changed event is published and consumed by auth + email + marketing systems (no failures/DLQ).
8. **Transactional Email Provider Contact**
    - Verify transactional provider contact/profile updated to new email (no duplicate contact with old email).
9. **Marketing/Promotional Sync**

    - After change, trigger promo/newsletter test send → ensure recipient is new email; old email unsubscribed/removed.

10. **Queued/Scheduled Campaign Safety**

- Schedule promo before change → change email → ensure send uses new email (or queued job is re-resolved at send time).

11. **Audit & Traceability**

- Check logs for reset/promo send → confirm it records: user id, email used, data source, timestamp.

12. **Security/Verification Edge**

- If email verification exists: change email → verify flow required before using new email for reset/promos.

Edge Cases

1. **Data Store Inconsistency**

    - The email address is updated in the primary profile database but not synchronized with the authentication/identity database used for password reset functionality.

2. **Cache Invalidation Failure**

    - Cached user records or email lookup keys are not properly invalidated, resulting in stale email data being used by downstream services.

3. **Read Replica Lag**

    - Services querying read replicas may retrieve outdated email information due to replication delay.

4. **Event Propagation Failure**

    - The Email Updated event is published but not successfully consumed by dependent services (e.g., email, marketing, authentication), possibly due to queue failures or dead-lettering.

5. **Pre-Queued or Scheduled Emails**

- Transactional or promotional emails were generated and queued prior to the email update and continue to be delivered to the old address.

6. **Marketing List Snapshot**

    - The marketing system uses a previously exported or snapshotted recipient list that still contains the old email address.

7. **Third-Party Email Provider Desynchronization**

    - The external Email Service Provider maintains a separate contact record that was not updated or contains duplicate entries for the user.

8. **Email Verification Dependency**

    - The system continues to use the last verified email address for certain communications if the newly updated email has not yet been verified.

9. **Multiple Accounts or Identity Conflicts**

    - The old email address is associated with a different or duplicate user account that is still active within certain systems.

10. **Email as Primary Identifier**

    - Some legacy services may treat email as a primary key, leading to inconsistent updates or creation of separate user records.

11. **Race Condition Between Update and Action**

    - A password reset or campaign trigger occurs concurrently with the email update, causing the system to read stale data.

12. **Multiple Email Fields**

    - Different services reference different email fields (e.g., login email vs. contact email), and not all fields were updated consistently.

13. **Multi-Region or Multi-Tenant Desynchronization**

    - In distributed environments, certain regions or tenants may not reflect the updated email due to delayed synchronization.

14. **Fallback or Business Logic Error**

    - System logic may incorrectly default to the previously stored email address under certain validation or delivery failure conditions.

Risk Assessment

1. **Unauthorized Account Access**
   If password reset links continue to be sent to the previous email address, there is a significant risk that an unintended recipient could gain unauthorized access to the user's account, leading to potential account compromise.

2. **Data Privacy and Confidentiality Breach**
   Sending transactional or promotional communications to an outdated email address may expose personal or account-related information to unauthorized individuals, resulting in a data privacy incident.

3. **Regulatory and Compliance Exposure**
   Failure to maintain accurate and up-to-date personal data may constitute non-compliance with applicable data protection regulations potentially leading to legal and financial consequences.

4. **Account Takeover Risk Due to Email Reassignment**
   If the old email address is reassigned to another individual, that party could exploit password recovery mechanisms to access the account.

5. **Customer Trust and Reputational Impact**
   Inconsistent handling of user contact information can erode customer confidence in the platform's reliability and security practices.

6. **Marketing and Consent Violations**
   Continuing to send promotional emails to an outdated address may violate user consent preferences and anti-spam regulations.

7. **System Integrity and Architectural Weakness**
   Data inconsistency across services indicates gaps in synchronization, event handling, or cache invalidation mechanisms, increasing the risk of broader system reliability issues.

8. **Operational and Incident Escalation Risk**
   If not promptly resolved, the issue may escalate into a cross-system production incident involving authentication, email services, and third-party providers.

5.

In a ride-sharing app, some rides start automatically without the driver clicking the “Start Trip” button.

This only happens when GPS signal is weak or when the driver switches between mobile data and Wi-Fi.

How would you recreate this bug?
What edge cases related to location tracking, background services, and network reconnect could be causing this behavior?

Ans:

Assumptions

1. Product & Functional Assumptions

- The ride-sharing driver application requires the driver to explicitly tap the Start Trip button to transition a ride into the Started state.
- Automatic trip initiation is not intended behavior under normal circumstances.
- The issue described (rides start automatically) indicates an unintended state transition occurring without explicit user interaction.
- The trip lifecycle likely includes defined states such as:
    - Assigned
    - Arrived at Pickup
    - Started / In Progress
    - Completed
- State transitions are expected to be governed by defined business rules and user-triggered events.

2. System Architecture Assumptions

- Trip state management involves both client-side logic and backend synchronization.
- The backend maintains the authoritative trip state, but the client may:
    - Optimistically update UI state
    - Queue actions when offline
    - Retry failed API requests upon reconnect
- The application uses real-time location tracking, potentially through:
    - GPS
    - Wi-Fi positioning
    - Cellular triangulation
    - Fused location providers (especially on Android)
- The app operates a background location service to continue tracking while minimized or when the device is locked.

3. Location Tracking Assumptions

- When GPS signal is weak:
    - Location accuracy radius increases.
    - Location updates may be delayed or stale.
    - Sudden location “jumps” may occur once signal improves.
- The system may rely on:
    - Distance thresholds from pickup location
    - Speed/movement detection
    - Geofence entry/exit events
- There may be logic that infers trip start conditions based on:

    - Sustained movement after arrival
    - Exiting pickup geofence
    - Matching route direction toward destination

4. Network & Connectivity Assumptions

- Switching between mobile data and Wi-Fi may cause:
    - Temporary loss of connectivity
    - Request timeouts
    - Duplicate API retries
    - Out-of-order request processing
- The application likely includes:
    - Automatic retry logic
    - Offline event queuing
    - Session/token refresh during reconnect
- Reconnection logic may replay pending events, potentially triggering unintended state transitions.

5. Background Service Assumptions

- The driver app continues running a location service while in the background.
- Background-to-foreground transitions may:
    - Reinitialize services
    - Trigger state reconciliation with backend
- The OS may:

    - Throttle background services
    - Restart killed services
    - Deliver delayed location updates in batches

6. Event & State Synchronization Assumptions

- Trip state updates may be triggered by:
    - Direct user action (button tap)
    - Location-based inference logic
    - Server-side processing of driver telemetry
- There may be safeguards such as:
    - A local boolean flag
    - Server-side validation checks
- The unintended behavior may result from:
    - Duplicate requests
    - Delayed event processing
    - Inconsistent client-server state synchronization
    - Misinterpretation of movement/location signals

7. Environment & Reproducibility Assumptions

- The issue is intermittent and likely timing-dependent.
- It may require a combination of:
    - Weak GPS signal
    - Network switching

    - App background/foreground transitions
    - Active trip in Arrived state
- The behavior could differ between platforms (Android vs. iOS) due to differences in background execution and location services.

Test ideas

1. Weak GPS Testing

- Go to a place where GPS signal is weak (like inside a building or basement).
- Check that the trip does NOT start automatically.
- Turn GPS on/off quickly and see if the trip starts by itself.
- Simulate wrong location jump (suddenly showing far away location) and check the trip should not start.

2. Network Switching Testing

- Switch between Wi-Fi and Mobile Data many times during Arrived state.
- Turn internet off for 10–20 seconds, then turn it on again.
- Check that the trip does NOT start automatically after reconnect.
- Test slow internet to see if retry requests cause auto start.

3. Background App Testing

- After reaching pickup, minimize the app.
- Move a little with weak GPS.
- Open the app again and check if the trip started automatically.
- Kill the app completely and reopen it. Make sure the trip is still not started.

4. Movement Testing

- After reaching pickup, move the car slowly without pressing Start Trip.
- Drive a short distance (100–200 meters).
- The trip should NOT start automatically.
- Test speed jump issue (GPS shows high speed suddenly). It should not trigger start.

5. Offline Testing

- Turn off internet before pressing start.
- Move around.
- Turn internet back on.
- Make sure the app does not auto start the trip after reconnect.

6. Duplicate Request Testing

- Use slow network so API request retries.
- Make sure multiple retries do not trigger trip start.
- Check server logs to confirm start reason is only USER_TAP.

7. App Restart Testing

- Restart the phone during Arrived state.
- Open the app again.
- Trip should still be in not started state.

8. Negative Testing (Very Important)

Trip should NEVER start automatically when:

- Driver did not press Start Trip

- GPS signal is weak
- Internet reconnects
- App is in background
- App restarts

9. Logs Checking

Whenever trip moves to Started, check:

- Who started it? (User tap or system?)
- Network type at that time
- GPS accuracy
- Any retry request happened?

6.

An online learning platform tracks video progress and unlocks certificates when 100% of the course is completed.

Many users report that their videos show fully watched, but certificates remain locked even after refreshing and re-logging.

What backend validations might be failing?
How would you verify event tracking, progress calculation, and completion triggers?

Ans: Assumptions Based on the Scenario

1. **UI progress may not be the final truth.**
   Just because the frontend shows 100% does not mean the backend has recorded 100% completion.

2. **Course completion may require more than watching videos.**
   There could be quizzes, assignments, or other requirements that must also be completed.

3. **Video watch data may not be fully sent to the server.**
   Due to network issues, browser restrictions, or event failures, the final watch events

might not reach the backend.

4. **The video completed event may not have been triggered.**
   If users skip to the end or close the tab quickly, the completion event might not be recorded.

5. **The certificate unlock process may have failed.**
   Even if progress reached 100%, the backend process that unlocks the certificate may not have executed properly.

Event Tracking Issue

- Final video completion event was not sent
- Events were rejected due to validation errors
- Wrong user ID or course ID was recorded
- Last few seconds of watch time were not captured

Progress Calculation Issue

- UI shows 100%, but backend calculation shows 99%
- Rounding differences in percentage calculation
- A required video or quiz is incomplete
- Progress from multiple devices was not merged correctly

Completion Trigger Issue

- Completion status was not updated to completed
- Certificate generation job failed
- Background job or queue processing did not run
- Completion trigger logic has a bug

How To Verify

Step 1: Check Event Tracking

- Review logs for the affected user
- Confirm the “video completed” event was received
- Check for any event validation errors\

Step 2: Check Backend Progress Calculation

- Verify the stored course progress percentage
- Confirm all required videos and quizzes are completed
- Recalculate progress manually if needed

Step 3: Check Completion & Certificate Trigger

- Verify course completion status in the database
- Check if certificate generation job was triggered
- Review logs for job failures or errors

Edge Cases

1 Video Watching Behavior Issues

- User skipped to the end instead of watching fully.
- User watched at 2x speed, and the system may require minimum watch time.
- User closed the tab/app immediately after the video ended, so the final event was not sent.
- Poor internet connection caused the last few seconds not to be recorded.
- User watched in background mode, and tracking stopped.
- Autoplay switched to next video too quickly, so the “video completed” event didn’t fire.

2. Frontend (UI) Showing Wrong Information

- The UI may show 100% based on local data, but the backend did not confirm it.

- The page may be showing cached (old) data.
- User refreshed before the backend finished updating.
- User opened the course in multiple tabs or devices, and progress didn't sync properly.

3. Event Tracking Problems

- The final "video completed" event was not received by the server.
- Events arrived in the wrong order and were rejected.
- Some tracking requests were blocked by ad blockers or browser settings.
- Wrong user ID or course ID was sent with the tracking event.
- Duplicate event handling removed an important completion event.

4. Progress Calculation Problems

- UI rounded progress to 100%, but backend shows 99%.
- A required quiz or assignment is incomplete.
- A new video was added to the course after the user started.
- A hidden required lesson exists that the user didn't complete.
- Progress from different devices was not merged correctly.

5. User Account or Access Issues

- User's enrollment expired.
- User logged in with a different account.
- Certificate is restricted for trial users.
- Organization rules prevent certificate generation.

6. Certificate Generation Problems

- Background job that generates the certificate failed.
- System marked course complete but failed to create certificate record.
- Queue or job processing is delayed.
- Certificate process is stuck in pending or failed state.

Risks in This Scenario

1. User Trust & Reputation Risk

If users complete a course but cannot access their certificate:

- It reduces trust in the platform.
- Users may feel the system is unreliable or unfair.
- Negative reviews and social media complaints may increase.
- Brand reputation may be damaged.

2.Customer Support & Operational Risk

- Increased support tickets and manual verification requests.
- Higher operational workload for support and engineering teams.
- SLA breaches if resolution time increases.
- Additional cost for manual certificate issuance.

3.Revenue & Business Risk

- Users may request refunds.
- Drop in renewals or subscription upgrades.
- Corporate clients may lose confidence.

- Reduced conversion rates for paid courses.

4. Data Integrity Risk

- Mismatch between frontend progress and backend records.
- Inaccurate reporting in analytics dashboards.
- Incorrect completion statistics affecting business decisions.
- Compliance or audit issues if certification data is incorrect.

5.Technical & System Reliability Risk

- Event tracking pipeline failures.
- Asynchronous job or queue backlogs.
- Race conditions or caching inconsistencies.
- Database replication lag causing inconsistent reads.

6. Compliance & Legal Risk

- Certificates may be required for professional or regulatory purposes.
- Incorrect issuance or failure to issue certificates can create legal disputes.
- Accreditation bodies may question system accuracy.

7.Scalability Risk

- As user volume increases, event processing delays may increase.
- Certificate generation system may not scale properly.
- Performance bottlenecks may become more frequent during peak hours.

8.Security & Fraud Risk

- Weak validation may allow users to bypass watch requirements.
- Strict validation may block legitimate users.
- Potential exploitation if completion logic has loopholes.

7.

A banking app processes money transfers and shows Transfer Successful instantly.

Later, some users complain that the receiver received the same amount twice, while their own balance was deducted only once.

What kind of transaction handling or retry mechanism could cause duplicate credits?
How would you test rollback, idempotency, and failure recovery?

Ans: Assumptions Based on the Scenario

1. Debit and Credit Are Done in Different Systems

The sender's balance is reduced in one system.The receiver's balance is increased in another system. They are not handled in the same database transaction.

2. Not a Single Atomic Transaction

Debit and credit do not happen at the exact same time in one safe transaction.They happen in separate steps.So if one step succeeds and another fails, inconsistency can happen.

3. Credit Happens Asynchronously

After debiting the sender, the system sends a message to a queue. A background worker later reads that message and credits the receiver. This delay can create problems if something fails in between.

4. System Has Automatic Retry

If the system does not receive a response (due to timeout or network issue), it retries automatically. But sometimes the first request already succeeded — only the response was lost. When it retries, the receiver may get credited again.

5. At-Least-Once Message Delivery

Most message queues guarantee at-least-once delivery. This means the same message can be delivered more than once. If there is no duplicate checking, the credit can happen multiple times.

6. No Proper Idempotency Check

There may not be a strong duplicate protection using:

- Unique transfer-id
- Unique idempotency key
- Database unique constraint

Without this, the system cannot detect repeated requests.

7. App Shows Success Too Early

The app shows Transfer Successful immediately after deducting money. But the receiver credit might still be processing in the background. So users think everything is finished, but actually it is not.

8. Possible Network or System Failures

Failures like:

- Network timeout
- Service crash
- Server restart
- Database failover

Test Ideas

1. **Duplicate Request Test**
   Send the same transfer request multiple times with the same transfer_id.

2. **Timeout and Retry Test**
   Simulate a timeout after credit is processed and verify retry does not create duplicate credit.

3. **Network Failure Test**
   Drop the response after successful credit and check if retry causes double credit.

4. **Duplicate Message Test**
   Send the same message/event from the queue more than once.

5. **Consumer Crash Test**
   Crash the worker after processing credit but before message acknowledgment.

6. **Service Restart Test**
   Restart services during transfer processing and verify no duplicate credit.

7. **Concurrency Test**
   Send multiple identical transfer requests at the same time.

8. **Partial Failure Test**
   Debit succeeds but credit fails — verify proper handling (no money loss or duplication).

9. **Database Constraint Test**
   Try inserting duplicate credit records with the same transfer_id.

10. **Reconciliation Test**
    Verify that for each transfer:

- Only one debit exists
- Only one credit exists
- Final balances are correct

Key Edge Cases

1. **Credit Success but Response Lost**
   Credit completed, but timeout caused retry → duplicate credit.

2. **Duplicate Message Delivery**
   Queue delivers the same credit event more than once.

3. **Consumer Crash After Processing**
   Worker crashes after credit but before acknowledgment → message reprocessed.

4. **Concurrent Processing**
   Multiple workers process the same transfer at the same time.

5. **Missing Idempotency Key**
   No unique `transfer_id` check → duplicate credit allowed.

6. **Partial Failure**
   Debit succeeds but credit fails → inconsistent state.

7. **Service Restart During Processing**
   System restarts and re-executes already completed steps.

8. **Multiple Retry Layers**
   Client + API + Worker all retry → duplicate execution.

Important Risks

1. **Financial Loss Risk**
   Duplicate credit can cause direct financial loss to the bank.

2. **Data Integrity Risk**
   Inconsistent debit and credit records break ledger accuracy.

3. **Reconciliation Failure Risk**
   System may fail during end-of-day settlement or audit checks.

4. **Customer Trust Risk**
   Incorrect balances reduce customer confidence in the banking system.

5. **Regulatory & Compliance Risk**
   Financial systems must meet strict regulatory standards. Duplicate transactions can lead to compliance violations.

6. **Fraud Exploitation Risk**
   Users may intentionally trigger retries to gain extra credit.

7. **Operational Risk**
   Manual correction effort increases workload for support and finance teams.

8. **System Stability Risk**
   Excessive retries and duplicate processing can overload services

8.

Push notifications arrive immediately for some users, but for others they arrive hours later or not at all.

This issue is inconsistent across devices, OS versions, and internet speeds.

How would you break down the testing across device state, background restrictions, notification services, and backend queues?

Ans: **Assumptions Based on the Scenario**

1. About the Problem

- Delayed means a notification arrives more than 5–15 minutes after it was sent from the backend.
- Not received means the user does not get the notification within a few hours, and there is no clear confirmation of delivery.
- The issue is related only to push notifications (not in-app messages or manual refresh).

2. Tracking and Logs

- Every notification has a unique ID so we can track it from backend to device.
- The backend logs:
    - When the notification was created
    - When it was added to the queue
    - When it was sent to the push service (FCM/APNs)
    - The response from the push service
- The app can log:
    - When the device token was created or refreshed
    - When the notification was received
    - Whether the app was in foreground, background, or closed

3. Device Token

- The device token stored in the backend is valid and up to date.
- If the user has multiple devices, the notification is sent to the correct one.
- Token refresh (after reinstall, update, or device change) works properly.

4. Push Notification Services (FCM / APNs)

- The system sends notifications using correct configuration (correct keys, topics, bundle ID).
- Notification payload size and format follow platform rules.
- Priority (high/normal) is set correctly.
- Push providers (FCM/APNs) may delay delivery in some cases.

5. Device State & Background Restrictions

- Users may have:
    - Battery saver mode ON
    - Background app refresh OFF
    - App force-closed
    - OEM battery restrictions (especially Android)
- Silent/data-only notifications may be delayed more than alert notifications.
- Notification permissions are enabled on the device.

6. Network Conditions

- Users may have:
    - Slow or unstable internet

    - High latency
    - VPN or firewall restrictions
- Internet speed alone does not guarantee fast delivery.

7. Backend Queue

- Notifications go through a queue before being sent.
- There may be:
    - Queue backlog
    - High traffic
    - Retry attempts with delay
- Each notification has a TTL (time limit). Expired notifications should not be delivered late.

8. Error Handling

- The system correctly handles:
    - Invalid tokens
    - Temporary push service errors
    - Permanent failures
- There are no silent failures without logging.

9. App Version & Environment

- The issue may affect specific app versions.
- Production and staging environments may have different configurations.
- Configuration mismatches may cause delivery problems.

10. User Settings

- Users may:
    - Disable notifications
    - Enable Do Not Disturb / Focus mode
    - Use notification summary (which delays display)
- Delivery and display are different things.

11. Investigation Goal

The goal is to find where the delay happens:

- Backend queue?
- Push provider?
- Device receipt?
- OS display?

Test Ideas – Push Notification Delay Issue

1. Device State Testing

1.1 App State Scenarios

Test notification delivery when the app is:

- In foreground
- In background
- Fully terminated (killed)
- After device restart
- After app reinstall

1.2 Screen & Lock State

- Device locked vs unlocked
- Screen ON vs OFF
- Device idle for long time (e.g., 2–3 hours)

2. Background & Battery Restrictions

2.1 Power Settings

Test with:

- Battery Saver ON / OFF
- Low Power Mode ON / OFF (iOS)
- Background App Refresh ON / OFF
- App added to battery optimization list (Android)

2.2 OEM Restrictions (Android)

Test on devices known for aggressive background killing:

- Xiaomi
- Oppo
- Vivo
- Samsung

2.3 Doze & App Standby (Android)

- Device idle for extended time
- App not opened for several days

3. Network Conditions Testing

3.1 Network Type

Test on:

- WiFi (strong signal)
- WiFi (weak signal)
- 4G / 5G
- Switching between WiFi and mobile data

3.2 Network Quality

Simulate:

- High latency
- Packet loss
- Intermittent connection
- No internet at send time, then reconnect

3.3 VPN / Firewall

- With VPN ON / OFF
- Corporate or restricted networks

4. Push Service Configuration Testing (FCM / APNs)

4.1 Priority & Payload

- High priority vs normal priority
- Silent/data-only notification vs alert notification
- Large payload vs small payload
- With and without collapse key

4.2 Token Validity

- Valid token
- Expired token
- Token after reinstall
- Token after OS upgrade

4.3 Multiple Devices

- Same user logged in on 2+ devices
- Verify correct device receives notification
- Disable notification on one device only

5. Backend Queue & Load Testing

5.1 Normal Traffic

- Single notification test
- Small batch (10–50 notifications)

5.2 High Load / Spike Testing

- Bulk send (thousands)
- Peak-hour simulation
- Verify queue delay time

5.3 Retry Mechanism

- Simulate temporary push service failure
- Verify retry interval

- Check exponential backoff logic
- Ensure no infinite retry loop

5.4 TTL (Time to Live)

- Short TTL test
- Expired notification should not be delivered late

6. Logging & Monitoring Validation

- Verify notification ID tracking end-to-end
- Compare:
    - Backend sent time
    - Push provider response time
    - Device receive time
- Check error logs for invalid tokens
- Ensure no silent failures

7. OS-Level Permission Testing

- Notifications permission granted / denied
- Notification channel disabled (Android)
- Focus mode / Do Not Disturb
- Scheduled notification summary (iOS)

8. Version & Environment Testing

- Different app versions (old vs latest)

- Production vs staging
- Different OS versions
- After app update

Edge Case Scenarios

- Device storage almost full
- Device date/time incorrect
- App force stopped
- User logs out before notification arrives
- User logs in after notification was sent

Validation Metrics

For each test, measure:

- Time from backend send → provider accept
- Provider accept → device receive
- Device receive → notification display

Classify issue source as:

- Backend delay
- Push service delay
- Device/OS restriction
- User settings issue

Risks – Push Notification Delay / Missing Notifications

1. Business Risks

1.1 Poor User Experience

- Users may miss important updates (orders, payments, alerts).
- Delayed notifications reduce trust in the app.
- Users may think the system is unreliable.

1.2 Customer Churn

- If notifications are critical (banking, delivery, OTP, health alerts), users may switch to competitors.
- Negative app store reviews may increase.

1.3 Revenue Impact

- Missed promotional notifications can reduce campaign effectiveness.
- Delayed transactional alerts may impact conversion rates.

2. Technical Risks

2.1 Backend Queue Overload

- High traffic may cause queue delays.
- Retry mechanisms may create message backlog.
- System may not scale properly during peak time.

2.2 Push Provider Throttling

- FCM/APNs may throttle or delay notifications.
- Misconfigured priority settings may cause low-priority delivery.

2.3 Invalid or Expired Device Tokens

- Sending to outdated tokens causes delivery failure.
- Token refresh issues after reinstall or OS upgrade.

2.4 Silent Failures

- Errors may not be logged properly.
- Notifications may fail without alerting the team.

3. Device & OS Risks

3.1 Background Restrictions

- Battery saver, Doze mode, or OEM restrictions may delay delivery.
- Silent notifications may not wake the app.

3.2 User Permissions

- Users may disable notifications.
- Notification channels may be turned off.
- Focus / Do Not Disturb may delay display.

3.3 OS Version Differences

- New OS updates may change background execution behavior.
- Some OS versions may handle push differently.

4. Network Risks

- Unstable internet connection.
- High latency or packet loss.
- VPN or firewall blocking push services.

- Temporary offline state at delivery time.

5. Data Consistency Risks

- Notification delivered late may contain outdated information.
- Duplicate notifications due to retry logic.
- Missing notifications for critical workflows (e.g., OTP).

6. Monitoring & Visibility Risks

- No end-to-end tracking of notification lifecycle.
- Difficult to identify where delay occurred (backend, provider, device).
- Lack of proper alerting for abnormal delay rates.

7. Compliance & SLA Risks

- If notifications are time-sensitive (OTP, security alerts), delay may break SLA.
- Regulatory impact in financial or healthcare systems.
- Security risk if fraud alerts are delayed.

9.

A user logs out of the mobile app and closes it completely.
When reopening the app later, they are automatically logged in again without entering credentials.

What session, token storage, or cache mechanisms could be responsible?
How would you test for security risks here?

Ans:Assumptions Based on the Scenario

1. Logout may not fully invalidate the session

It is assumed that the logout function may only clear the UI state but does not revoke the server-side session or invalidate authentication tokens. If the backend still considers the session valid, the user can be logged in automatically.

2. Refresh token is still active

The application may use long-lived refresh tokens. If the refresh token is not deleted or revoked during logout, the app can silently generate a new access token when reopened.

3. Tokens are stored in persistent secure storage

Authentication tokens may be stored in device secure storage. These storage systems persist even after the app is closed, allowing automatic session restoration.

4. App closure does not clear authentication data

Force-closing the app only terminates the process. It does not remove stored tokens, cookies, cached credentials, or local database flags such as `isLoggedIn = true`.

5. Cookies or SSO session may still exist

If the app uses WebView or Single Sign-On , login cookies may remain active. As a result, reopening the app restores the session without asking for credentials.

Security Testing Approach

To identify security risks, the following tests should be performed:

1. **Verify token invalidation:**
   After logout, attempt to reuse the old access token to access protected APIs. If access is still granted, logout is incomplete.

2. **Check refresh token behavior:**
   Monitor network traffic after reopening the app. If a refresh token is used successfully, it was not properly cleared.

3. **Inspect local storage:**
   Confirm whether tokens remain stored on the device after logout.

4. **Test replay attack scenario:**
   Attempt to reuse captured tokens from another device or session. If successful, it indicates weak session control.

Test Ideas

1. Logout Functionality Validation

Objective: Verify whether logout properly terminates the user session.

Test Steps:

1. Log in with valid credentials.
2. Perform logout from the application.
3. Completely close (terminate) the app.
4. Reopen the application.

Expected Result:
The user should be redirected to the login screen and required to authenticate again (as per defined security policy).

2. Access Token Invalidation Test

Objective: Ensure access tokens are invalidated after logout.

Test Steps:

1. Log in and capture the access token using a proxy tool .
2. Perform logout.
3. Attempt to access a protected API endpoint using the previously captured token.

Expected Result:
The API should respond with 401 Unauthorized or equivalent error.
If access is still granted, it indicates improper token invalidation.

3. Refresh Token Revocation Test

Objective: Confirm that refresh tokens are revoked during logout.

Test Steps:

1. Log in and perform logout.
2. Monitor network traffic upon reopening the app.

3. Observe whether a refresh token request is automatically triggered.

Expected Result:
Refresh token should not successfully generate a new access token after logout.
Successful refresh indicates that the token was not properly revoked.

4. Secure Storage Verification

Objective: Validate that authentication data is removed from device storage after logout.

Test Steps:

1. Log in and then logout.

2. Inspect secure storage locations.

Expected Result:
All authentication tokens, session identifiers, and sensitive user data should be cleared.
Persistence of tokens indicates a security weakness.

5. Cookie / SSO Session Validation (If Applicable)

Objective: Verify that WebView or SSO session cookies are cleared during logout.

Test Steps:

1. Log in using WebView or SSO authentication.

2. Perform logout.

3. Reopen the login screen.

Expected Result:
The user should not be automatically authenticated through existing cookies or browser sessions.

6. Multi-Device Session Test

Objective: Evaluate whether logout invalidates the session across devices (if required by policy).

Test Steps:

1. Log in on Device A.
2. Capture authentication token.
3. Logout from Device A.
4. Attempt to use the captured token on Device B or via API request.

Expected Result:
The token should be invalidated and rejected by the server.

7. Session Persistence vs Logout Comparison

Objective: Differentiate between expected session persistence and improper logout behavior.

Test Steps:

1. Log in and force close the app without logging out.
2. Reopen the application.

Expected Result:
User may remain logged in if the session is still valid.
This confirms whether the issue is related specifically to logout implementation.

8. Token Expiry Handling Test

Objective: Ensure expired tokens cannot be reused.

Test Steps:

1. Log in and wait until the access token expires.
2. Attempt to use the expired token or reopen the application.

Expected Result:
Expired tokens must not grant access. The application should require secure re-authentication or valid refresh flow.

Edge Cases

1. Internet / Network Issues

- User logs out when internet is slow or disconnected.
- Logout request does not reach the server.
- App shows logged out but session is still active on backend.

2. App Closed Too Quickly

- User taps logout and immediately force closes the app.
- Logout process may not complete properly.

3. Token Not Properly Removed

- Access token still stored in the phone.
- Refresh token still saved in secure storage.
- App uses saved token to log in automatically.

4. Expired Access Token but Valid Refresh Token

- Access token expires.
- Refresh token is still active.
- App silently generates a new access token and logs in user again.

5. Remember Me Feature

- User selected Remember Me.
- Logout does not override this setting.
- App keeps user logged in.

6. SSO / Google / Apple Login

- User logged in using Google or Apple.

- Logout from app does not log out from Google/Apple session.
- App auto-logs in using existing SSO session.

7. Multiple Devices

- User logs out from Device A.
- Same session still active on Device B.
- Token may still work if reused.

8. Background App Refresh

- App refreshes session in background.
- Even after logout, app may request new token automatically.

9. Biometric Login Enabled

- Face ID / Fingerprint login enabled.
- After logout, app still allows quick unlock without full credentials.

10. Cached User Data

- App saves isLoggedIn = true in local storage.
- On reopen, app checks local flag instead of server validation.

Risks

1. Unauthorized Access Risk

If logout does not fully end the session, anyone who gets access to the device can open the app and access the user’s account without entering credentials.

Impact:
Sensitive personal or financial information may be exposed.

2. Token Misuse Risk

If access tokens or refresh tokens are not properly revoked after logout, they can still be used to access protected APIs.

Impact:
An attacker who captures the token (via proxy, malware, or rooted device) can reuse it even after logout.

3. Session Hijacking Risk

If the server does not invalidate sessions during logout, the same session ID may remain active.

Impact:
Attackers may continue using the active session without the user's knowledge.

4. Replay Attack Risk

If old tokens still work after logout, attackers can replay captured tokens from another device or environment.

Impact:
Unauthorized remote access to user accounts.

5. Shared Device Risk

On shared or public devices, another person can reopen the app and access the previous user's account.

Impact:
Privacy violation and potential data breach.

6. Compliance & Regulatory Risk

Improper logout behavior may violate security standards such as:

- OWASP Mobile Security Guidelines
- GDPR (data protection principles)
- PCI-DSS (if financial data involved)

Impact:
Legal issues, compliance failure, and reputational damage.

7. Loss of User Trust

If users expect logout to fully end their session but the app logs them in automatically, it reduces confidence in the app's security.

Impact:
Negative user experience and potential customer loss.

8. Privilege Escalation Risk

If user role or permissions change on the backend but cached session still allows old permissions, unauthorized actions may be possible.

Impact:
Access to restricted features or sensitive operations.

10.

An e-commerce platform allows users to change currency (USD, EUR, BDT).

After switching currency, product listing prices update correctly, but checkout total remains in the old currency — causing users to pay wrong amounts.

What integration gaps between frontend, backend pricing engine, and payment system might exist?

Ans: Assumption based on this scenario

Key assumptions

1. **Currency switch only updates what the user sees in the product listing UI.**
   The listing page is using the selected currency to display converted prices correctly.

2. **Checkout total is not recalculated using the same selected currency.**
   Checkout is still using a cached value, an old cart total, or a backend value created before the currency change.

3. **There is a mismatch between "display currency" and "transaction currency."**
   The UI may show one currency, but the backend/payment request is still using another.

4. **The system has at least two sources of truth for price:**
    - One for product listing display (frontend or a pricing API)
    - Another for cart/checkout totals (backend cart service, pricing engine, or order service)
5. **Payment provider is charging based on what the backend sends—not what the UI displays.**
   Payment gateways typically charge the exact amount + currency code they receive.
6. **Currency selection is stored somewhere (cookie/session/local storage/user profile), but not consistently shared across services.**
   The selected currency may not be carried into cart, checkout, and payment steps.

Likely integration gaps (frontend ↔ backend pricing ↔ payment)

1) Frontend doesn't send currency to checkout/cart APIs

- Listing uses selected currency locally (or a different API),
- But add-to-cart / cart / checkout API calls don't include `currency=EUR` (for example).
  Result: backend continues pricing in USD.

2) Backend cart stores prices as "final numbers" without currency context

- Cart line items might store `unit_price=100` but not store `currency=USD`.
- Or store it once and never refresh it.
  Result: totals are computed from old currency prices.

3) Pricing engine is not invoked again after currency change

- Listing calls pricing/FX conversion live,
- Checkout uses cart snapshot totals without re-pricing.
  Result: checkout total stays in the old currency.

4) Currency is applied only as a UI conversion (display-only)

- Frontend converts prices visually using FX rates,

- Backend still treats original USD prices as the truth.
  Result: user sees EUR, but order/payment is still USD.

5) Caching problems (frontend or backend)

- Cart/checkout totals are cached per session/user,

- Cache key doesn't include currency (missing `currency` dimension).
  Result: switching currency doesn't invalidate cached totals.

6) Order service uses different rules than listing (tax, shipping, rounding)

- Listing might show just product price,

- Checkout total includes taxes/shipping/discounts and is calculated in a different service.
  Result: currency conversion is done in one place but not the other.

7) Payment request uses old currency/amount

- Backend creates a payment intent/charge in USD,

- UI changes currency afterward but payment intent is not updated/recreated.
  Result: gateway charges the old currency.

8) Missing validation before payment

- System doesn't verify:

  - order currency == selected currency

  - displayed total == payable total
    Result: mismatch reaches payment successfully.

Important Test Ideas (Currency Mismatch Scenario)

1. Currency Propagation Test

- After switching currency (USD → EUR → BDT), verify the updated currency appears on:
    - Product listing
    - Product details page
    - Cart
    - Checkout
- Currency must remain consistent across all pages.

2. Cart Recalculation Test

- Add products in one currency, then switch to another.
- Verify:
    - Line item prices update
    - Discount updates
    - Tax and shipping recalculate
    - Grand total updates correctly

3. Backend Validation Test

- Check that cart/checkout API sends correct currency parameter.
- Backend response must include correct:
    - Currency code
    - Unit price
    - Total amount

4. Cache Validation Test

- Switch currency multiple times.

- Ensure no old totals are reused (cache must consider currency).

5. Payment Integration Test (Critical)

- Verify payment request contains:
    - Correct amount
    - Correct currency
- Payment gateway charge must exactly match checkout total.

6. Edge Case Test

- Test with:
    - Coupons (percentage & fixed discount)
    - Shipping selection
    - Tax calculation
    - Rounding differences

Important Risks

1. Financial Loss Risk

If checkout total remains in old currency, customers may pay less or more than the actual amount.This can cause direct revenue loss or refund costs.

2. Customer Trust & Reputation Risk

Mismatch between displayed price and charged amount reduces customer trust.It may lead to complaints, bad reviews, and brand damage.

3. Payment Dispute / Chargeback Risk

Customers may raise disputes if charged in a different currency than shown.This increases chargebacks, penalty fees, and payment gateway risk rating.

4. Compliance & Legal Risk

Displaying one currency and charging another may violate consumer protection laws in some regions.
This can create legal exposure.

5. Data Integrity Risk

If frontend, backend, and payment systems use different currency values, order records may become inconsistent.This affects accounting, reporting, and reconciliation.

6. Caching & Stale Data Risk

If totals are cached without currency context, users may see incorrect amounts.This leads to unpredictable checkout behavior.

7. Pricing Engine Integration Risk

If currency change does not trigger full repricing (items, tax, shipping, discount), system calculations become unreliable.

8. Operational & Support Risk

Increased customer complaints create extra workload for support and finance teams.

11.

A movie ticket booking system allows two users to reserve the same seat at the exact same time during peak hours.

Both users receive confirmation messages, but only one can enter the hall.

How would you test concurrency control, database locking, and real-time seat availability?

Ans: Assumption based on scenario

Important Assumptions

1. **Each seat for a showtime is unique in the database.**
   One seat (e.g. A10 for 7 PM show) can belong to only one user.

2. **Booking is confirmed only after database commit.**
   A success message must be sent only when the transaction is fully saved.

3. **Two users can click Book at the exact same time.**
   The system must handle true simultaneous requests.

4. **Database is the single source of truth.**
   Entry to the hall is validated using the same booking record stored in the database.

5. **There is a unique constraint on (showtime_id + seat_id).**
   The database must prevent duplicate bookings technically.

6. **Seat has clear status values.**
   Available → Held → Booked.

7. **System may run on multiple servers.**
   Concurrency control must rely on database locking, not only application logic.

Test Ideas for Seat Double Booking Scenario

1. Simultaneous Booking Test

Purpose: Check race condition handling.

- Two users try to book the same seat at the exact same time.
- Expected Result:
  - Only one booking succeeds.
  - The second user receives “Seat not available.”
  - Only one record exists in the database.

2. High Traffic Test

Purpose: Verify system behavior under heavy load.

- Many users (100+) attempt to book the same seat simultaneously.
- Expected Result:
  - Only one successful booking.

    - All others fail properly.
    - No duplicate records.
    - System remains stable.

3. Database Unique Constraint Test

Purpose: Ensure database prevents duplicate booking.

- Verify unique constraint on (showtime_id + seat_id).
- Try creating duplicate booking manually or via API.
- Expected Result:
    - Database rejects duplicate entry.
    - No two confirmed bookings for same seat.

4. Transaction Locking Test

Purpose: Validate proper database locking.

- Start booking from User A and delay commit.
- User B tries booking same seat during delay.
- Expected Result:
    - User B waits or gets failure.
    - Only one booking is confirmed.

5. Real-Time Seat Update Test

Purpose: Ensure seat availability updates instantly.

- Open seat map for two users.
- User A books seat.

- Expected Result:
    - Seat becomes unavailable immediately for User B.
    - User B cannot complete booking.

6. Seat Hold Expiry Test

Purpose: Verify hold mechanism works correctly.

- User selects seat but does not complete payment.
- Wait for hold time to expire.
- Expected Result:
    - Seat becomes available again.
    - No confirmed booking exists.

7. Duplicate Request Test (Double Click)

Purpose: Prevent duplicate booking from repeated requests.

- Same user clicks confirm button multiple times quickly.
- Expected Result:
    - Only one booking is created.
    - No duplicate confirmation.

8. Confirmation Validation Test

Purpose: Ensure confirmation is sent only after successful booking.

- Verify confirmation message is triggered after database commit.
- Expected Result:
    - Every confirmation has a matching database record.

    - No false confirmation is sent.

Edge Cases

1) Double Click / Rapid Submit

- User clicks Confirm multiple times quickly.
- **Expected:** Only one booking is created, no duplicate confirmation.

2) Network Timeout + Retry

- Booking request succeeds in backend, but user gets timeout and retries.
- **Expected:** System returns same booking or blocks duplicate booking.

3) Slow Internet / Stale Seat Map

- User sees seat as Available because UI is not updated yet.
- **Expected:** Backend must reject booking if seat already taken.

4) Payment Success but Seat Already Booked

- Payment is successful, but another user already booked the seat at the same time.
- **Expected:** System must not confirm booking; refund/rollback process should trigger.

5) Payment Failed After Hold

- Seat was held, but payment failed.
- **Expected:** Seat returns to Available, and no confirmed booking is stored.

6) Hold Expiry While User Is Paying

- User takes too long; hold expires during payment.

- **Expected:** Booking should fail or re-check availability before confirm.

7) App Server Crash During Booking

- Server crashes after user clicks confirm but before database commit.
- **Expected:** No partial booking; transaction rolls back cleanly.

8) Database Deadlock / Lock Timeout

- Two transactions lock resources in a way that causes deadlock.
- **Expected:** One transaction fails safely and system retries or shows clear message.

9) Duplicate Messages / Notification Issue

- Booking fails, but user still receives confirmation SMS/email due to async delay.
- **Expected:** Notifications must be sent only after successful commit.

10) Hall Entry Validation Mismatch

- User has confirmation screen but booking record is missing or cancelled.
- **Expected:** Entry system must validate against database and reject invalid ticket.

11) Multiple Tabs / Multiple Devices

- Same user tries booking from two browser tabs/devices at same time.
- **Expected:** Only one booking is allowed; others should fail or be blocked.

12) Seat Release After Cancel

- User cancels booking, but seat still shows unavailable for others.
- **Expected:** Seat becomes available immediately and database status is updated.

Important Risks in This Scenario (Seat Double Booking)

1) Double Booking Risk (Main Risk)

- Two users may reserve the same seat due to concurrency issue.
- **Impact:** Customer anger, revenue loss, reputation damage.

2) False Confirmation Risk

- System sends confirmation to both users even though only one booking is valid.
- **Impact:** Trust breaks, support tickets increase, refund requests increase.

3) Data Integrity Risk

- Duplicate booking records may exist in database if constraints/locking are weak.
- **Impact:** Reports, seat availability, and ticket validation become unreliable.

4) Real-Time Sync Risk (Stale Seat Availability)

- Seat map shows “Available” even after seat is booked by someone else.
- **Impact:** Users waste time selecting seats and face failure at payment/confirm step.

5) Payment and Refund Risk

- Payment succeeds but booking fails (or seat already booked).
- **Impact:** Financial disputes, refund delays, chargebacks.

6) Retry / Duplicate Request Risk

- Due to timeout or double click, same user request may execute multiple times.
- **Impact:** Duplicate bookings, duplicate charges, confusion.

7) Hold/Timeout Mismanagement Risk

- Seat hold expires incorrectly or does not release seat properly.

- **Impact:** Seats get blocked unnecessarily, revenue loss.

8) Deadlock / Lock Timeout Risk

- High traffic can cause database deadlocks or long waits.
- **Impact:** Slow system, booking failures, poor user experience.

9) System Crash During Booking Risk

- App crash between payment and booking commit can create inconsistent state.
- **Impact:** Paid but no ticket, or seat locked forever.

10) Entry Validation Mismatch Risk

- Gate system may allow entry based on wrong data, or reject valid users.
- **Impact:** Hall management issues, security problems, customer complaints.

11) Scalability Risk

- Under peak hours, system may slow down and increase race conditions.
- **Impact:** More booking errors and loss of sales.

12) Audit & Compliance Risk

- Booking logs and payment logs may not match due to async processing.
- **Impact:** Difficult to investigate issues and settle disputes.

12.

Uploading small images works perfectly, but uploading large images freezes the app without any error message.

No crash logs appear on the server.

How would you test memory usage, API limits, timeout behavior, and client-side handling for large payloads?

Ans: Assumption based on scenario

Important Assumptions

1. The application freeze is occurring on the client side, not due to a server crash, since no server crash logs are generated.

2. Large image files require significantly more memory and processing power, which may cause the UI thread to become unresponsive.

3. The client application may be loading the full image into memory (for preview, encoding, or conversion) before uploading, increasing memory usage.

4. The image upload is likely being sent as a single large request, not in chunks, which increases the risk of timeout or memory issues.

5. There may be a maximum request body size limit configured at the API server, load balancer, CDN, or reverse proxy level.

6. Large uploads may exceed client-side or server-side timeout limits, causing the request to fail silently.

7. The request might be getting blocked or dropped before reaching the main server application, which explains why no error logs are visible.

8. The application may not have proper error handling or user feedback mechanisms for large payload failures.

9. Image decoding or processing on either the client or server may temporarily use more memory than the file size itself, leading to performance degradation.

Test Ideas for Large Image Upload Freeze Issue

1. Memory Usage Testing

- Monitor client-side memory usage while uploading small vs large images.

- Test on low-memory devices to check if the app becomes unresponsive.

- Upload extremely large images (e.g., 20MB, 50MB, 100MB) and observe behavior.
- Check for memory spikes during image preview, encoding, or resizing.

2. API Request Size Limit Testing

- Verify the maximum allowed request body size on:
    - API server
    - Load balancer
    - Reverse proxy
    - CDN (if used)
- Upload files just below and just above the limit.
- Confirm that proper error responses are returned.
- Validate that the client shows a clear error message.

3. Timeout Behavior Testing

- Simulate slow network conditions (3G, high latency).
- Upload large files over slow internet and monitor:
    - Client-side timeout
    - Server-side timeout
    - Idle connection timeout
- Check if the app shows timeout errors properly.
- Verify retry functionality (if available).

4. Client-Side Handling Testing

- Ensure the UI remains responsive during upload.
- Verify that a loading indicator or progress bar is shown.
- Test cancel upload functionality.
- Confirm that errors are properly displayed to the user.
- Test background upload behavior (minimize app, switch tabs).

5. Server-Side Processing Testing

- Check if large images cause high CPU or memory usage on the server.
- Monitor logs for partial uploads or aborted connections.
- Validate image processing performance (resize/compression).

6. Payload Handling Testing

- Test different image formats (JPG, PNG, HEIC, WebP).
- Test images with high resolution but compressed size.
- Test base64 encoded uploads (if used) vs multipart upload.
- Test corrupted or partially uploaded files.

Edge Cases for Large Image Upload

1. File size just below and just above the allowed limit.
2. Very high-resolution image even if file size looks normal.
3. Different image formats (JPG, PNG, HEIC, WebP).
4. Corrupted or partially damaged image file.
5. Non-image file renamed as .jpg or .png.
6. Very long file name or special characters (spaces, emoji, symbols).

7. Uploading multiple images at the same time, including one large file.

8. Upload over very slow internet connection.

9. Internet disconnection during upload.

10. Switching network (Wi-Fi to mobile data) while uploading.

11. Minimizing the app or switching tabs during upload.

12. Cancelling the upload in the middle of the process.

13. Server returning Payload Too Large.

14. Uploading on a low-memory or low-storage device.

15. Rotating the screen while upload is in progress.

Risks in Large Image Upload Scenario

1. **Poor User Experience**
   If the app freezes without showing an error, users may think the app is broken.

2. **User Drop-off / Loss of Trust**
   Repeated freezing can reduce user trust and increase uninstall rate.

3. **Data Loss Risk**
   User may lose selected image or progress if the app becomes unresponsive.

4. **High Memory Usage Risk**
   Large images may consume too much RAM and cause crashes on low-end devices.

5. **Server Overload Risk**
   Large uploads can increase server CPU and memory usage.

6. **API Limit Risk**
   If request size exceeds API limit, uploads will fail without proper handling.

7. **Timeout Risk**
   Large files may exceed client or server timeout limits.

8. **Network Instability Risk**
   Slow or unstable internet may increase failure rate.

9. **Security Risk**
   Large or malformed files could be used to test system limits (DoS risk).

10. **Infrastructure Configuration Risk**
    Different limits at CDN, load balancer, and server may cause inconsistent behavior.

11. **Scalability Risk**
    If many users upload large files simultaneously, system performance may degrade.

12. **Error Handling Gap**
    Lack of proper validation and user feedback may hide real issues.

13.

A form submission works perfectly in Chrome and Edge but fails silently in Safari — no error shown, no data stored.

How would you approach cross-browser debugging?
What front-end and network differences might cause this?

Ans: Assumption based on scenario

Assumptions

- The form works correctly in Chrome and Edge, meaning the backend API and database are functioning properly.
- In Safari, the form submission fails silently — no success message, no error message, and no data stored.
- The issue is likely browser-specific, not a server-side problem.
- The frontend may be using JavaScript (fetch/AJAX) for submission.
- Safari's stricter handling of cookies, CORS, security policies, or modern JavaScript features may be causing the failure.

Test Ideas (Simple Version)

- Submit the form with minimum required fields.
- Try with long text and special characters.

- Test in Safari normal mode and private mode.
- Open Safari DevTools → Network tab and check:
    - Is the request sent?
    - What is the status code?
- Compare Safari request with Chrome request.
- Check if cookies/session are sent in Safari.
- Look for CORS or preflight (OPTIONS) errors.
- Check if there is any redirect (302/307).
- Disable cache and test again.
- Check server logs to see if Safari request reaches backend.

Important Edge Cases to Test (Safari Form Submission)

- **Empty required fields** – Does validation work properly?
- **Very long input values** – Large text in textarea or input fields.
- **Special characters & emoji** – `@, #, %, &, <, >,` 😊
- **Different date formats** – Non-ISO date vs ISO format.
- **File upload cases** – Large file, empty file, unsupported format.
- **Slow internet connection** – Does request timeout or fail silently?
- **Session expired** – Does Safari handle 401/403 properly?
- **Third-party cookies blocked** – If API is on different domain.
- **Private browsing mode** – Storage/cookies may be restricted.
- **Redirect responses (302/307/308)** – Does Safari follow correctly?

- **CORS preflight failure (OPTIONS request)** – Missing headers.
- **Double click submit** – Multiple requests sent?
- **Cache issue** – Old response reused instead of new request.
- **JavaScript disabled or partially blocked** – Does form still work?

Risks in This Scenario (Safari form fails silently)

- **Data loss risk:** Safari users submit the form but nothing is saved, so leads/orders/registrations are lost.
- **Bad user experience:** No error message → users think it worked, then get frustrated later.
- **Revenue/business impact:** If this form is checkout/lead/contact, it can directly reduce sales and conversions.
- **Trust damage:** Users may stop using the site/app because it doesn't work on iPhone/Safari.
- **Hidden production issue:** Silent fail means monitoring may not catch it quickly; issue stays live longer.
- **Compliance risk:** If it's a legal/consent form, missing submissions can create audit/compliance problems.
- **Support cost increases:** More complaints, manual follow-ups, and extra customer support workload.
- **Platform bias in analytics:** Metrics look fine in Chrome but Safari segment drops, giving misleading insights.
- **Security/auth risk:** If caused by cookies/SameSite/ITP, users may be logged out or requests may be unauthenticated.
- **Data integrity risk:** Partial submissions or duplicate clicks can create duplicate records or inconsistent data.
- **Release risk:** Future updates may worsen cross-browser compatibility if polyfills/error handling aren't added.

- **Reputation risk:** Public reviews/comments like doesn't work on iPhone can harm brand image.

14.

Users report that their order history sometimes disappears completely, but reappears after refreshing or logging out and back in.

What caching layers, API inconsistencies, or sync issues could cause this ghost-data behavior?

Ans: Assumption based on scenario

Possible Causes of "Ghost" Order History Issue

Assumptions

- Order history data comes from backend API.
- System uses caching (browser/app/server level).
- Authentication is token/session based.
- Multiple services are involved (orders, user profile, database, etc.).

Main Possible Reasons

1. Client-Side Cache Issue

Sometimes the app/browser stores an empty response in cache.If a temporary error happens, the system may save an empty list and show it. After refresh or re-login, the app fetches fresh data and the order history reappears.

2. Server or CDN Caching Problem

If personalized data (like order history) is cached incorrectly, users may receive a wrong or empty response.Refreshing or logging in again may bypass the cache and show correct data.

3. Authentication / Session Mismatch

If there is a token or session issue, the order service may not recognize the user properly. Instead of showing an error, it might return an empty list.Re-login refreshes the session and fixes the issue.

4. Database Sync or Replication Delay

If the system uses read replicas or search indexing, there might be a delay in syncing data.During that delay, the order history may appear empty.After some time or refresh, data becomes available again.

5. Pagination or API Bug

If the wrong page number or expired cursor is used, the API may return no data. Refreshing resets pagination and loads the correct results.

Test Ideas for Order History Disappears and Reappears Scenario

1. Client-Side Cache Testing

1.1 Refresh Behavior

- Verify difference between normal refresh and hard refresh.
- Check if order history reappears after a hard reload.
- Compare UI data with actual network response in developer tools.

1.2 Cache Clearing Test

- Clear browser/app cache and test again.
- Test in Incognito/Private mode.
- Verify behavior after app reinstall (for mobile).

1.3 Offline/Online Mode

- Open order history while offline.
- Reconnect to network and verify if auto-sync works properly.

2. API Response Validation Testing

2.1 Empty Response Handling

- Simulate API returning HTTP 200 with an empty array.
- Verify whether the UI incorrectly shows “No orders” instead of handling errors properly.

2.2 Error Handling

- Simulate API errors (500, 503, timeout).
- Check if the system converts errors into empty lists.

2.3 Consistency Testing

- Send multiple API requests for the same user.
- Verify if responses are consistent across calls.

3. Authentication & Session Testing

3.1 Expired Token Scenario

- Test behavior when authentication token expires.
- Verify whether silent token refresh works properly.

3.2 Logout/Login Flow

- Check if logout clears cached data.
- Verify if login triggers fresh data fetch.

3.3 Multi-Device Login

- Login with the same account on multiple devices.
- Verify consistency of order history.

4. Database & Synchronization Testing

4.1 Read-After-Write Test

- Place a new order.
- Immediately check order history.
- Measure delay before order appears.

4.2 Replica/Region Testing

- Verify if data differs across different servers or regions.

5. Pagination & State Management Testing

5.1 Page Reset

- Navigate to page 2 or later.
- Refresh and check whether page resets correctly.

5.2 Cursor Expiration

- Test behavior when pagination token expires.
- Verify system handles invalid cursor correctly.

6. Cross-Platform Testing

- Test across different browsers (Chrome, Firefox, Safari).
- Test on mobile (Android, iOS).
- Compare web vs mobile behavior.

7. Load & Stress Testing

- Test during high traffic.
- Simulate multiple concurrent requests.
- Verify system consistency under load.

Key Risks in the Ghost Order History Scenario

1. **Loss of Customer Trust**
   Users may believe their orders are lost or deleted, reducing confidence in the system.

2. **Increased Support Cost**
   Missing order history can generate more customer complaints, tickets, and operational workload.

3. **Duplicate Orders & Financial Risk**
   Customers may place duplicate orders or request refunds/chargebacks if they cannot see previous purchases.

4. **Data Consistency Issues**
   Indicates potential caching, synchronization, or database replication problems, affecting system reliability.

5. **Security & Privacy Risk**
   Incorrect caching or session handling could expose another user's order data, leading to serious compliance issues.

6. **Monitoring & Detection Gaps**
   If errors are returned as empty responses, system failures may go unnoticed in monitoring tools.

15.

A referral program rewards users for inviting friends.

Some users manage to receive referral bonuses multiple times for the same person by slightly changing emails or timing registrations.

How would you design tests to catch fraud cases and validation loopholes?

Ans: Assumptions based on scenario

Assumptions Based on the Scenario

1. The referral program gives a bonus when a user invites a friend.
2. The bonus is given after the referred user completes a required action (such as sign-up, verification, or first purchase).
3. The system may currently identify users mainly by email address.
4. Small changes in email are treated as new users.
5. The system may not properly prevent duplicate payouts.
6. There may be timing or event-processing issues that allow the same referral to be rewarded multiple times.
7. Strong identity validation (phone, device, payment method) may not be fully enforced.

1) Email manipulation tests (same person, different email text)

Goal: Stop multiple bonuses using email tricks.

Test ideas

- Case change: `User@x.com` vs `user@x.com`
- Spaces: `user@x.com` vs `user@x.com`
- Plus alias: `user+1@gmail.com` vs `user@gmail.com`
- Dot alias (gmail): `u.ser@gmail.com` vs `user@gmail.com`
- Unicode lookalike: `user@gmail.com` vs `user@gmail.com`

Expected

- System normalizes email and blocks duplicate referral bonus.

2) Same person using different identity signals

Goal: Catch “new account” fraud with changed email.

Test ideas

- Same phone, new email → should not pay again
- Same device fingerprint, new email → flag/hold
- Same payment card/bank, new email → block duplicate bonus
- Same address + same name (fuzzy match) → flag

Expected

- Bonus paid only once per real person (identity cluster).

3) Double payout / idempotency tests

Goal: Prevent bonus being paid twice due to retries or replays.

Test ideas

- Trigger “reward payout” API twice with same inputs
- Run payout job twice (retry simulation)
- Send purchase/verification webhook twice
- Duplicate message in queue (at-least-once delivery)

Expected

- Only one payout record exists.
- Second attempt returns already processed without payment.

4) Timing / race condition tests

Goal: Prevent exploiting registration timing.

Test ideas

- Referral cookie set → signup after 30 mins
- Signup happens first → referral cookie arrives later (should not re-assign)
- Two signups at the same time using same referral
- Referred user changes device/browser mid-flow

Expected

- Referral attribution is fixed at a clear point.
- No duplicate reward from event ordering.

5) Self-referral tests

Goal: Block users referring themselves.

Test ideas

- Same device + same IP + same payment method between referrer and referred
- Same phone number used by both accounts
- Same KYC identity (if applicable)

Expected

- Self-referral is blocked or sent to manual review.

6) Abuse / volume pattern tests

Goal: Detect suspicious referral behavior.

Test ideas

- One referrer creates 30 referred accounts in 10 minutes
- Many referrals from same IP block / same device farm

- Very high referral success rate compared to normal users
- Many referred accounts do first action instantly (bot-like speed)

Expected

- Rate limit triggers, bonus held, fraud flag raised.

7) Cross-referrer conflict tests

Goal: One referred user should not generate bonus for multiple referrers.

Test ideas

- Same referred user clicks two referral links (A then B)
- Referral link shared in public group and reused
- Referral code applied after signup

Expected

- Clear rule enforced (first-touch or last-touch), only one winner.

8) Account lifecycle edge-case tests

Goal: Stop loopholes using account deletion/changes.

Test ideas

- Referred user signs up, deletes account, signs up again → bonus should not repeat
- Referred user changes email after signup → should not create new eligibility
- Referrer changes code after referred user signup

Expected

- Eligibility tied to stable identity, not mutable fields.

9) Data integrity / audit tests

Goal: Ensure investigations are possible.

Test ideas

- Verify logs capture: referrer_id, referred_id, normalized email, phone hash, device id, ip, timestamps

- Verify dashboards show duplicate attempts and blocked payouts

Expected

- Enough traceability to prove and debug fraud cases.

Edge Cases

1.Email Edge Cases

- Same email but capital letter different
- Email with space before/after
- Gmail “+” trick (user+1@gmail.com)
- Gmail dot trick (u.ser@gmail.com)
- Fake or temporary email

2.Tracking Edge Cases

- User clicks referral link on mobile but signs up on desktop

- User opens link in incognito mode

- User clicks referral link A, then link B

- User signs up many days after clicking referral link

3.Duplicate Event Edge Cases

- Signup request sent twice

- Purchase event received twice

- System retry job runs again
- User double-clicks payment button

4.Same Person, Multiple Accounts

- Same phone number, different emails
- Same device, different emails
- Same payment method, different emails
- Same address with small changes

5.Self-Referral Edge Cases

- User creates second account using own referral link
- Same device/IP used for both accounts
- Same payment method used

6.Reward Rule Problems

- User signs up but does not verify
- User makes purchase then cancels/refunds
- Very small or fake purchase to trigger bonus
- Bonus given for every purchase instead of only first

7.Account Delete & Recreate

- User signs up → deletes account → signs up again
- User changes email after signup

8. High Volume Fraud

- One user refers many accounts in short time
- Many accounts from same IP
- Very fast signup and purchase (bot behavior)

Risks in This Referral Program Scenario

1. Financial Risk

- Same person receives bonus multiple times.
- Duplicate payouts due to system errors.
- Fake referrals increase company cost.

2. Fraud Risk

- Users create multiple fake accounts.
- Self-referral (user rewards themselves).
- Organized referral farming using bots or device farms.

3.System / Technical Risk

- Duplicate events (webhook replay, job retry).
- Race conditions during signup or payout.
- Missing idempotency in reward processing.

4 .Validation Loophole Risk

- Email variations bypass duplicate checks.
- Weak identity validation (only email checked).
- No strong cross-check with phone, device, or payment method.

5. Reputation Risk

- Honest users see fraud happening.
- Trust in referral program decreases.

- Brand image damage.

6. Compliance / Audit Risk

- No proper logging of referral and payout data.
- Hard to investigate fraud cases.
- Possible financial reporting issues.

7. False Positive Risk

- Legitimate users from same household/device get blocked.
- Real referrals wrongly marked as fraud.

16.

After a major app update, only long-time users experience crashes during checkout, while new users work fine.

What backward-compatibility risks would you investigate?
How could old data formats or migrations cause this?

Ans: Assumption based on scenario

A major application update has been recently released, potentially introducing changes to the data model, APIs, business logic, or local storage structure.

The issue occurs specifically during the checkout flow, which typically involves complex interactions between user profile data, saved payment methods, addresses, cart state, promotions, taxes, and backend services.

The crashes are reported only by long-time users, while newly registered users complete checkout successfully.

The problem is likely client-side, as it results in application crashes rather than standard transaction failures or server-side validation errors.

Long-time users likely possess legacy persisted data, such as:

- Old local database schemas
- Cached objects or serialized models
- Saved payment tokens
- Deprecated feature flags
- Outdated cart or profile structures

The update may have included data migrations, which may not fully account for all legacy data states.

New users operate on a clean data environment, created entirely under the updated schema and logic, thereby avoiding legacy compatibility issues.

The checkout flow may trigger specific logic paths that rely on previously stored user data, exposing incompatibilities or assumptions that do not hold for older data formats.

The crashes are likely tied to edge cases in legacy data, such as missing fields, deprecated enum values, nullability changes, or incompatible data types that were not fully addressed during migration.

Test Ideas

1. Upgrade & Migration Testing

- Test upgrade paths from multiple older app versions to the latest version.
- Validate local database migrations using real legacy data.
- Simulate interrupted or partial migrations.

- Verify migrations are idempotent and handle skipped versions.

2. Legacy Data Validation

- Test deserialization of old cart, profile, and payment data structures.
- Validate handling of missing, null, deprecated, or unexpected fields.
- Test unknown or outdated enum values.

3. Saved Payment Methods

- Validate compatibility of legacy payment tokens with updated SDK/API.
- Test expired cards, deprecated providers, and missing required fields.
- Ensure graceful validation errors instead of crashes.

4. Cart & Checkout Logic

- Test legacy cart structures and outdated promotions.
- Validate recalculation of totals with old pricing formats.
- Test products that are no longer available.

5. API Contract Compatibility

- Validate handling of older user profiles missing newly required fields.
- Test API response changes (added/removed fields).
- Verify versioning behavior.

6. Feature Flags & Experiments

- Test users with legacy feature flags or experiment assignments.
- Validate fallback behavior for deprecated flags.

Edge Cases to Validate (Checkout crashes only for long-time users)

- **Skipped-version upgrades:** users updating from very old versions miss intermediate migrations.
- **Partial/failed migrations:** app terminated during migration → inconsistent local DB/state.
- **Non-idempotent migrations:** migration runs twice → duplicate rows, broken constraints, invalid references.
- **Legacy serialization:** old cached JSON/objects can't be decoded (missing/renamed fields, type changes).
- **Unknown legacy enums:** deprecated values for payment/shipping/discount types not handled by new code.
- **Nullability changes:** fields now assumed required (postal code, phone, region, currency) are null in old profiles/addresses.
- **Saved payment incompatibility:** old payment tokens/SDK metadata invalid for the new payment provider flow.
- **Keychain/keystore schema changes:** stored auth/payment secrets can't be read/decrypted after update.
- **Legacy cart contents:** discontinued SKUs, old bundle/variant formats, or stale pricing objects crash recalculation.
- **Old promotions/loyalty:** deprecated coupon rules or points precision changes cause negative/overflow/assert failures.
- **Feature flags/experiments drift:** long-time users stuck in deprecated buckets or cached flags route to broken paths.
- **Large historical data:** many addresses/cards/orders → memory/performance spikes during checkout assembly.
- **Concurrency/timing:** double-submit, background/foreground during payment redirect, or network switching corrupts state.

Key Risks in This Scenario

1. Backward Compatibility Risk

Legacy user data (profiles, carts, payment tokens, addresses) may not conform to the new data model, causing runtime exceptions during checkout.

2. Migration Failure Risk

Incomplete, skipped, or non-idempotent database migrations may leave long-time users with inconsistent or corrupted local state.

3. Serialization/Deserialization Risk

Changes in field names, data types, or enum values may cause decoding failures when loading persisted legacy objects.

4. Payment Integration Risk

Stored payment methods or old token formats may be incompatible with updated payment SDKs or API contracts.

5. Nullability & Validation Risk

Fields previously optional but now required (e.g., postal code, tax ID, region) may be missing in older accounts, leading to crashes.

6. Business Logic Regression Risk

Changes in pricing, tax calculation, promotions, or loyalty logic may not account for legacy cart structures or historical discounts.

7. Feature Flag & Experiment Drift

Long-time users may be tied to deprecated experiments or cached feature flags, triggering unsupported checkout flows.

8. Data Volume & Performance Risk

Older users typically have larger datasets (order history, multiple addresses/cards), increasing memory usage and crash probability.

9. API Contract Mismatch Risk

Backend responses may assume new required fields that do not exist in legacy user records.

10. Reputation & Revenue Risk

Checkout crashes directly impact revenue, customer trust, retention, and app store ratings—especially among loyal customers.

17.

A real-time analytics dashboard updates live numbers, but occasionally the values jump backward instead of increasing.

How would you test timestamp handling, data streaming order, and sync delays?

Ans: Assumption based on scenario

1. Timestamp Handling

- Validate whether metrics use event time or processing time.
- Test with clock skew, late-arriving events, and day/time boundary cases.
- Verify watermark or lateness logic does not cause finalized values to decrease unexpectedly.
- Ensure every update includes an as of timestamp or version to prevent older data overwriting newer data.

2. Data Streaming Order

- Simulate **out-of-order events** and confirm the system correctly reorders or handles them.
- Test duplicate and replayed events to ensure idempotent aggregation.
- Validate that updates carry sequence/version numbers, and the UI ignores stale updates.

3. Sync Delays & System Lag

- Simulate network delays, WebSocket reconnects, and replica lag.
- Check that slower snapshots do not overwrite more recent streamed values.
- Measure end-to-end latency (event → display) and monitor rollback frequency.

Test Ideas:

1. Timestamp Tests

- Send old events after new events → check if number goes backward
- Change system time (clock mismatch) → verify calculation stays correct
- Test around midnight/hour change → make sure totals don't reset wrongly

2. Data Order Tests

- Send events in wrong order → check final total
- Send duplicate events → make sure no double counting
- Restart system and replay data → number should stay correct

3. Sync / Delay Tests

- Add network delay → make sure old data doesn't replace new data
- Disconnect and reconnect dashboard → values should stay correct
- Simulate database lag → no backward jump

4. Simple Check

- For cumulative numbers (like total sales), verify they never decrease

Edge cases

Timestamp Edge Cases

- Events with future timestamps
- Events with very old timestamps
- Multiple events with same timestamp

- Daylight Saving Time (DST) change
- Events exactly at midnight / hour boundary
- Missing or null timestamp
- Incorrect time zone format

Data Order Edge Cases

- Events arriving completely out of order
- Same event sent multiple times (duplicates)
- Partial batch delivery (half received, half delayed)
- Consumer restart during processing
- Partition/key change mid-stream

Sync / Delay Edge Cases

- Snapshot arrives after newer live updates
- WebSocket reconnect and receives older data
- Database replica lag
- Cache not refreshed but stream updated
- Network drop during update

Load / Performance Edge Cases

- Very high event volume
- Sudden traffic spike
- Very low traffic (long idle gap)

- System auto-scaling during stream

UI Edge Cases

- Browser refresh during update
- Multiple tabs open with same dashboard
- Client device clock incorrect
- Rapid page switching

Risks

1. Business Risks

- **Loss of trust** – Users may think data is incorrect or unreliable.
- **Wrong decisions** – Managers may make decisions based on unstable numbers.
- **Revenue impact** – Incorrect sales/transaction data can affect reporting.
- **Compliance risk** – Financial or audit reports may become inaccurate.

2. Technical Risks

- Data inconsistency between systems (stream vs database).
- Duplicate or missing data due to replay or ordering issues.
- Clock synchronization problems across services.
- Snapshot overwriting live data (stale data issue).
- Event processing failures during high load.

3. User Experience Risks

- Confusing dashboard behavior (numbers decreasing unexpectedly).

- Increased support tickets.
- Reduced confidence in analytics platform.
- Misinterpretation of real-time trends.

4. Operational Risks

- Difficult root cause analysis due to distributed system complexity.
- Monitoring gaps (no alert when rollback happens).
- Scaling issues under traffic spikes.
- Cache or replica lag causing inconsistencies.

5. Data Integrity Risks

- Incorrect aggregation logic.
- Late-arriving events modifying finalized results.
- Lack of idempotency in event processing.
- No version control in UI updates.

18.

A chat app shows messages as “Delivered,” but the receiver never gets them.

What message acknowledgment flows would you test to ensure true delivery vs UI assumptions?

Ans: Assumption based on scenario

The Delivered status is triggered after the message is successfully accepted and processed by the backend server, rather than after confirmation from the recipient’s device.

There is no mandatory device-level acknowledgment (recipient app receipt confirmation) required before updating the sender's UI to Delivered.

Acceptance of push notifications by third-party services may be incorrectly treated as proof of successful delivery to the recipient.

The system may rely on optimistic UI updates without validating end-to-end message receipt.

There may be a gap between message storage (database persistence) and actual message synchronization to the recipient's active session or device.

Failure scenarios may not properly trigger a fallback status such as Failed or Not Delivered.

Timeout or retry mechanisms for undelivered messages may be missing or not properly reflected in the UI.

The current message status logic may represent backend-level success rather than true end-to-end delivery confirmation from sender device to recipient device.

Test Ideas – Message Shows Delivered but Not Received

1. **Verify Delivered Trigger Point**
   Confirm whether Delivered is triggered by server acceptance or by recipient device acknowledgment.

2. **Online Recipient Test**
   Recipient connected via WebSocket. Ensure Delivered appears only after recipient device sends receipt ACK.

3. **Offline Recipient Test**
   Recipient offline during send. Status should remain Sent/Queued and change to "Delivered" only after recipient comes online and syncs.

4. **Push Notification Validation**
   Ensure push acceptance (FCM/APNs success) does not automatically mark message as Delivered.

5. **Network Failure Scenario**
   Simulate fanout failure after server acceptance. Verify system does not incorrectly show

Delivered.

6. **Multiple Device Scenario**
   Test delivery when recipient has multiple devices. Validate business rule (delivered to one vs all devices).

7. **Timeout & Retry Handling**
   If recipient acknowledgment is not received within defined time, system should retry or mark as Not Delivered.

8. **Permission/Block Scenario**
   If recipient blocks sender or is removed from chat, ensure status is not incorrectly marked as Delivered.

Edge Cases

1. Recipient is offline when message is sent.
2. Recipient turns on internet later but message does not sync.
3. Recipient app is closed / force stopped.
4. Recipient is logged out.
5. Internet connection is weak or unstable.
6. Server saves the message but fails to send it to recipient.
7. Push notification is sent but recipient does not actually receive/open it.
8. Recipient has multiple devices and message reaches only one device.
9. Recipient blocks the sender.
10. Recipient is removed from group before delivery.
11. Message is sent twice due to retry but not received properly.
12. Attachment/message media upload fails but shows delivered.
13. System delay or server overload causes delivery lag.

14. Message delivered but cannot be opened due to app error.

Risks :

User Trust Risk
Users lose trust in the app if messages appear delivered but are not actually received.

Communication Failure Risk
Important or time-sensitive messages may be missed, causing personal or business impact.

Data Integrity Risk
Mismatch between UI status and actual system state indicates unreliable message tracking.

Reputation Risk
Frequent delivery issues can damage brand credibility and user retention.

Escalation & Support Cost Risk
Increased customer complaints, tickets, and operational overhead.

Business Risk
For enterprise or transactional chats, incorrect delivery status may cause financial or legal issues.

Technical Risk
Hidden system flaws (ack handling, retry logic, push handling) may affect scalability and reliability.

19.

A user cancels a subscription, gets a cancellation confirmation email, but still has premium access for weeks.

What billing system, permission sync, or cron job failures might exist?

Ans: Assumption based on the scenario

1. Billing System Considerations

a) Cancellation Effective at Period End (Expected Behavior)

Many subscription systems interpret cancellation as disabling auto-renewal while maintaining access until the end of the prepaid billing period. In such cases:

- The subscription status becomes *cancelled at period end*.
- The user retains access until current period end.

If the user prepaid monthly or annually, continued access for weeks may be correct behavior.

b) Incorrect Effective Date or Proration Logic

- Miscalculation of `access end at` due to timezone discrepancies or date logic errors.
- Proration/refund workflows failing, leaving the subscription in a paid-through state.
- Grace period logic incorrectly extending access.

c) Duplicate or Parallel Subscriptions

- The user may have multiple active subscriptions.
- Cancellation applied to one subscription, while another remains active.
- Duplicate customer records causing mismatched updates.

2. Entitlement / Permission Synchronization Failures

In many systems, billing status and user permissions are decoupled. Access is controlled by an entitlement service that updates asynchronously via webhooks, message queues, or background workers.

Potential Failures:

- Billing webhook not delivered (network/TLS/misconfiguration).
- Webhook delivered but not processed.
- Idempotency or event versioning conflicts preventing state updates.
- Incorrect mapping between billing customer ID and internal user ID.
- Cached entitlements not invalidated after cancellation.
- Downstream authorization services not refreshed after entitlement changes.

In this case, billing may correctly reflect cancellation while permissions remain unchanged.

3. Scheduled Jobs / Cron Failures

If access revocation relies on periodic batch jobs:

- Revocation cron job not running (scheduler failure or disabled job).
- Pagination or batching logic skipping certain users.
- Timezone or date comparison errors delaying revocation.
- Job failures halting processing mid-batch.
- Shard/region-specific processing gaps.

A malfunction in expiration processing can result in extended premium access beyond the intended end date.

4. Authentication / Session Management Issues

If entitlement information is embedded in authentication tokens:

- Long-lived JWTs may contain stale premium claims.
- Client applications may not refresh tokens after cancellation.
- Server authorization may rely solely on token claims instead of real-time entitlement checks.

- Mobile or offline clients may delay state synchronization.

Access may persist until token expiration or user reauthentication.

5. Manual Overrides or Feature Flags

- Support teams may have manually granted premium access.
- Promotional or compensatory entitlements may supersede billing state.
- Feature flag rules may inadvertently include the user.

In such cases, billing cancellation does not automatically revoke manually assigned access.

6. Third-Party Marketplace Subscriptions

For subscriptions managed via Apple App Store, Google Play, or resellers:

- Cancellation may not have been completed at the platform level.
- Server-to-server notifications may be delayed.
- Platform subscription may still be active despite internal system updates.

Test Ideas

1. Cancellation Effective Date Validation (Critical)

Objective: Confirm that access duration aligns with defined business rules.

- Verify whether cancellation is configured as:
    - Immediate termination, or
    - Cancel-at-period-end.
- Validate that `current_period_end` matches the entitlement `expires_at`.
- Confirm premium access is removed precisely at the expected time.

2. Billing-to-Entitlement Synchronization

Objective: Ensure billing events correctly update user permissions.

- Trigger subscription cancellation.
- Confirm webhook/event delivery from billing provider.
- Verify entitlement record updates in the database.
- Confirm access reflects updated entitlement status in real time.

3. Webhook Failure & Retry Handling

Objective: Validate system resilience to integration failures.

- Simulate webhook endpoint downtime.
- Confirm retry mechanisms function correctly.
- Validate no permanent entitlement inconsistency occurs.
- Ensure failed events are logged and recoverable.

4. Expiration & Revocation Job Reliability

Objective: Confirm scheduled processes revoke access on time.

- Allow subscription to reach expiration.
- Verify revocation cron/job executes successfully.
- Confirm no users are skipped due to batching, pagination, or shard logic.
- Validate timezone handling for expiry comparison.

5. Duplicate or Parallel Subscription Handling

Objective: Ensure cancellation applies to all relevant active subscriptions.

- Create multiple subscriptions for the same user (e.g., web + mobile).

- Cancel one subscription.
- Confirm access persists only if another valid subscription exists.
- Validate correct customer-to-user ID mapping.

6. Authentication & Token Validation

Objective: Prevent stale session-based premium access.

- Issue authentication token with premium claim.
- Cancel subscription.
- Attempt access using existing token.
- Confirm server validates entitlement dynamically rather than relying solely on cached claims.
- Verify token refresh updates permissions correctly.

7. Manual Override & Entitlement Precedence

Objective: Validate hierarchy of entitlement sources.

- Apply manual premium grant.
- Cancel subscription.
- Confirm system correctly handles precedence rules (billing vs. manual).
- Ensure cancellation does not unintentionally preserve premium through alternate entitlement paths.

8. Audit, Logging & Monitoring Verification

Objective: Ensure observability and traceability.

- Validate logging of:
    - Cancellation event receipt

    - Entitlement update
    - Revocation job execution
- Confirm alerting for:
    - Webhook processing failures
    - Cron job downtime
    - Billing-entitlement mismatches

Edge Cases

1. **Cancel-at-period-end (expected)**

- Cancel disables auto-renew only; access remains until `current_period_end`.

2. **Cancel near renewal cutover**

- Cancel happens minutes around renewal → race condition (renew+cancel) → wrong end date.

3. **Proration/refund path partially fails**

- Email sent, refund job fails, subscription stays paid-through → premium stays.

4. **Duplicate subscriptions / multi-platform**

- Web cancelled but Apple/Google still active (or two plans active: add-on + base).

5. **Wrong user mapping**

- Billing customer/subscription linked to different internal user (email change, account merge).

6. **Webhook not received / not processed**

- Endpoint down, queue backlog, consumer crash, DLQ poison message → entitlement not updated.

7. **Out-of-order / duplicate events**

- Renewal event processed after cancel → entitlement re-granted; duplicates break idempotency.

8. **Revocation cron/job failure**

- Expiry scan job not running / paging bug / shard skip → access not revoked on time.

9. **Timezone/DST date math bug**

- `expires_at` computed in wrong timezone → extends by days/weeks.

10. **Token/cache staleness**

- Long-lived JWT/session claims `premium=true`, cache TTL too long, no invalidation on cancel.

11. **Manual override / promo precedence**

- Support comp, promo, org seat overrides billing cancellation → premium correctly persists but reason unclear.

Risks:

1. Revenue Leakage (High Risk)

- Users may continue consuming premium services without paying.
- Scales significantly if caused by systemic sync or revocation failure.
- Direct financial loss and inaccurate revenue reporting.

2. Billing–Entitlement Inconsistency (High Risk)

- Mismatch between billing system and access control system.
- Creates data integrity issues.
- Complicates audits, reconciliation, and financial compliance.

3. Customer Trust & Reputation Risk (High Risk)

- Users may be confused about whether cancellation worked.
- Risk of disputes, chargebacks, or complaints.
- Damages credibility of billing transparency.

4. Operational Risk (Medium–High)

- Support workload increases due to unclear subscription state.
- Manual investigations required (billing logs, webhook logs, entitlement DB).
- Engineering time spent on reconciliation and incident handling.

5. Security & Abuse Risk (Medium)

- Users may intentionally exploit cancellation timing to extend premium access.
- Potential for shared accounts or misuse of premium features without valid payment.

6. Compliance & Audit Risk (Medium)

- Financial reporting may reflect cancelled subscriptions while service remains active.
- Potential SOX/compliance concerns if subscription states are inconsistent.

7. Monitoring & Observability Gap (Medium)

- If issue goes undetected, indicates lack of alerting for:
    - Webhook failures
    - Cron job failures
    - Entitlement mismatch
- Suggests systemic reliability weakness.

20.

After changing password successfully, the user can still log in using the old password for several minutes.

What cache layers, session tokens, or delayed invalidation might be causing this serious security issue?

Ans: Assumption based on the scenario

1. Authentication Result Caching

It is assumed that the system may cache authentication results or user credential data (e.g., password hash or validation outcome) at the application or distributed cache level.

If such cache entries are not immediately invalidated upon password change, the old password may continue to be treated as valid until the cache TTL (time-to-live) expires. This is a common cause when the delay window is consistent.

2. Database Replication Lag (Read-After-Write Inconsistency)

Another strong assumption is that the password update is written to the primary database, while login validation reads from a read replica.

If replication is asynchronous, a temporary inconsistency may occur where the replica still holds the old password hash for several minutes. During this lag window, authentication using the old password would still succeed.

This is particularly likely in distributed or multi-region deployments.

3. Distributed System Cache Without Proper Invalidation

In multi-instance deployments, each application node may maintain an in-memory cache of user credentials or user profile data.

If password changes do not trigger cross-node cache invalidation, some nodes may continue validating against stale data until their local cache expires.

This would explain intermittent behavior depending on which server instance processes the login request.

4. Incomplete Session and Token Revocation

If the authentication system uses JWTs, refresh tokens, or persistent sessions, it is possible that password changes do not invalidate:

- Existing access tokens
- Refresh tokens
- Server-side sessions
- Remember me cookies

In such cases, the user may appear to be logging in with the old password, when in fact the system is still accepting previously issued tokens that remain valid until expiration.

If token revocation is not enforced upon password change, this represents a significant security gap.

5. Asynchronous or Event-Driven Password Update Workflow

It is assumed that the password update process may be asynchronous.

If the system confirms success to the user before all authentication services are fully updated, a temporary validation gap may exist.

Test Ideas to Investigate Delayed Password Invalidation

1. Fresh Session Login Test (Token Elimination Test)

Objective: Determine whether old session tokens are being reused.

Steps:

- Change the password.
- Open a new private/incognito browser session.
- Clear all cookies, local storage, and session storage.

- Attempt login using the old password.

2. Repeated Login Timing Test (TTL Detection)

Objective: Identify whether a fixed cache TTL is involved.

Steps:

- Change password.
- Immediately attempt login with the old password.
- Retry every 30–60 seconds.
- Record the exact time when old password stops working.

3. Multi-Instance / Load Balancer Test

Objective: Detect node-level cache inconsistency.

Steps:

- Change password.
- Attempt login with old password multiple times.
- Capture server instance identifiers (if available via headers/logs).
- Compare behavior across different instances.

4. Replica Lag Validation Test

Objective: Confirm read-after-write inconsistency.

Steps:

- Change password.
- Immediately query the user record directly from:

    - Primary database
    - Read replica
- Compare password hash values.

5. API-Level Authentication Test

Objective: Ensure login endpoint is not caching responses.

Steps:

- Call login API directly using a tool like Postman or curl.
- Disable client-side caching.
- Add cache-control headers if possible.
- Attempt login with old password.

6. Token Revocation Test (JWT / Refresh Tokens)

Objective: Verify that password change invalidates all tokens.

Steps:

- Login and obtain access + refresh token.
- Change password.
- Attempt:
    - Using existing access token
    - Refreshing access token
    - Calling protected endpoints

Edge Cases – Old Password Still Works After Password Change

1. New Browser / Incognito Test

After changing the password, try logging in with the old password using a private/incognito window.

- If it still works, the issue is likely related to caching or database replication.
- If it does not work, the issue may be related to session or token reuse.

2. Multiple Devices Test

Change the password on Device A and immediately try logging in with the old password on Device B.

- If the old password works on another device, there may be synchronization or replication delay.

3. Multiple Browser Test

Change the password in one browser and test the old password in a different browser.

- If behavior differs, session or cookie handling may not be properly invalidated.

4. Timing Window Test

After changing the password, attempt login with the old password every 30–60 seconds.

- If it stops working after a fixed time , this likely indicates cache TTL expiration.

5. Load Balanced Environment Test

If the system runs on multiple servers, attempt multiple login requests after password change.

- If the old password works intermittently, some server instances may still have cached credentials.

6. Refresh Token / Remember Me Test

Login with Remember Me enabled. Change the password. Close and reopen the browser.

- If access is still granted without re-authentication, session or refresh tokens are not being revoked.

7. API vs UI Test

Test login through both the web UI and direct API calls.

- If one path rejects the old password and the other accepts it, authentication logic may be inconsistent.

8. Cross-Region Test (If Applicable)

If the application is deployed in multiple regions, change the password in one region and test login from another.

- If the old password works in one region only, cross-region replication delay may be the cause.

9. Admin Password Reset Test

If an administrator resets the password, test whether the old password still works.

- If it does, session invalidation may not be triggered for admin-initiated changes.

10. Concurrent Request Test

Attempt to log in with the old password at the same time the password is being changed.

- If intermittent success occurs, there may be a race condition in the authentication flow.

Risk Assessment

1. Security Control Failure

Password change is a critical security control intended to immediately revoke the validity of the previous credential.
If the old password continues to work, this indicates a breakdown in authentication integrity and access control enforcement.

2. Continued Unauthorized Access Risk

If an attacker has knowledge of the old password, they can maintain access during the delay window.

Impact:

- Ongoing account compromise
- Unauthorized transactions or actions
- Data exposure

Even a short delay significantly increases exploitation risk.

3. Incident Containment Weakness

Users typically change passwords after suspected compromise.
If the old password remains valid, the system fails to immediately contain the threat.

This undermines the effectiveness of the password change as a recovery mechanism.

4. Session and Token Management Risk

The issue may indicate that:

- Active sessions are not invalidated
- Refresh tokens are not revoked
- Authentication caches are not cleared

This exposes the system to persistent access risk even after security-sensitive actions.

5. Data Protection and Compliance Exposure

For systems handling sensitive or regulated data (PII, financial, healthcare), delayed credential invalidation may:

- Violate internal security policies
- Conflict with industry best practices
- Increase regulatory and audit risk

6. Reputational Risk

Users expect password changes to take effect immediately.
Failure to enforce this can reduce trust and damage brand credibility.

7. Elevated Risk for Privileged Accounts

If administrative or high-privilege accounts are affected, the impact could include:

- System-wide configuration changes
- Data manipulation or deletion
- Service disruption